COURS COMPLET

Git, GitHub & GitLab

et leurs outils : Git Bash · GitHub Desktop · GitHub Actions · GitLab CI/CD · GitKraken

Orienté data science, data analytics et développement — explications, exemples testés, 54 exercices corrigés et TP final

Version du 08/11/2026

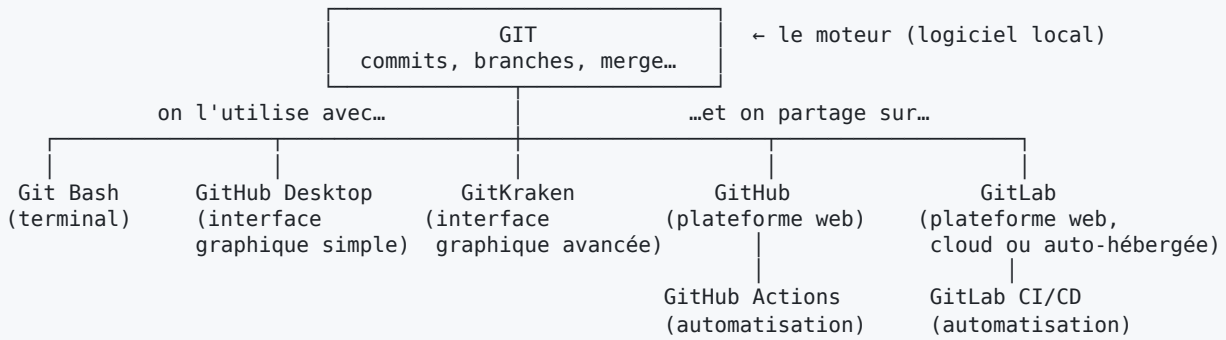
Sommaire

1. 🎓 Cours complet : Git, GitHub, GitLab et leurs outils
2. Chapitre 1 — Git : le moteur de tout le reste
3. Chapitre 2 — Git Bash : un terminal Linux sous Windows
4. Chapitre 3 — GitHub : héberger et collaborer
5. Chapitre 4 — GitHub Desktop : Git sans ligne de commande
6. Chapitre 5 — GitHub Actions : automatiser (CI/CD) sur GitHub
7. Chapitre 6 — GitLab : la plateforme DevOps tout-en-un
8. Chapitre 7 — GitLab CI/CD : pipelines d'automatisation
9. Chapitre 8 — GitKraken : le client Git graphique avancé
10. Chapitre 9 — TP final : un projet data de A à Z
11. Annexe — Code source complet des exemples

🎓 Cours complet : Git, GitHub, GitLab et leurs outils

Cours en français, orienté **data science / data analytics / développement**, avec explications, exemples exécutables, **exercices corrigés à la fin de chaque chapitre** (54 au total) et un TP final.

Comment s'articulent ces outils ?



Outil	Catégorie	En une phrase
Git	Logiciel de versionnement	Enregistre l'historique de ton code, localement
Git Bash	Terminal (Windows)	Un shell Bash + Git sous Windows
GitHub	Plateforme d'hébergement	Héberge tes dépôts, Pull Requests, Issues
GitHub Desktop	Client graphique	Git par boutons, simple, centré sur GitHub
GitHub Actions	CI/CD	Exécute tests et déploiements automatiquement sur GitHub
GitLab	Plateforme d'hébergement DevOps	Équivalent de GitHub, installable en interne, Merge Requests
GitLab CI/CD	CI/CD	Pipelines définis dans `.gitlab-ci.yml`
GitKraken	Client graphique avancé	Graphe visuel, rebase et conflits en glisser-déposer, multi-plateforme

📚 Chapitres

1. Git — les fondamentaux
2. Git Bash — le terminal sous Windows
3. GitHub — héberger et collaborer
4. GitHub Desktop — Git sans ligne de commande
5. GitHub Actions — CI/CD sur GitHub
6. GitLab — la plateforme DevOps
7. GitLab CI/CD — pipelines
8. GitKraken — client graphique avancé
9. TP final — un projet data de A à Z

Exercices : chaque chapitre se termine par une section 📝 *Exercices* puis une section 🔑 *Corrigés* (sur une nouvelle page dans la version PDF). Fais les exercices avant de regarder les corrigés ; ceux qui portent sur Git, Bash et les fichiers YAML ont été vérifiés en les exécutant.

Ordre conseillé : 1 → 2 → 3 → 4 → 5, puis 6 → 7, puis 8 et 9. Si tu travailles uniquement avec GitLab, tu peux faire 1 → 2 → 6 → 7 → 8 → 9.

🧪 Exemples fournis (testés)

Fichier	Contenu
`exemples/projet-demo/`	Mini-paquet Python (`moyenne` , `mediane` avec gestion des valeurs manquantes) + tests pytest
`exemples/github-actions/ci.yml`	Workflow GitHub Actions : lint + tests sur 3 versions de Python + artefact de couverture
`exemples/github-actions/rapport-hebdo.yml`	Workflow planifié (cron) avec secret — modèle pour un rapport data
`exemples/gitlab-ci/.gitlab-ci.yml`	Pipeline GitLab : stages, rules, cache, matrice, couverture, déploiements staging/production
`exemples/gitignore-data-science`	`.gitignore` prêt à l'emploi pour un projet data

> Les fichiers CI sont rangés dans `exemples/` pour **ne pas s'exécuter** dans ce dépôt. Pour les activer dans ton propre projet : `ci.yml` → `.github/workflows/ci.yml` , et `.gitlab-ci.yml` → racine du dépôt (voir le TP, chapitre 9).

Lancer les tests du projet de démo :

```
cd cours-git/exemples/projet-demo
pip install -r requirements-dev.txt
python -m pytest --cov=demo
```

🗒️ Aide-mémoire Git

Besoin	Commande
Configurer son identité	`git config --global user.name "Nom"` / `user.email "email"`
Créer / cloner un dépôt	`git init` / `git clone <url>`
Voir l'état	`git status` , `git diff` , `git diff --staged`
Préparer / enregistrer	`git add <fichier>` / `git commit -m "message"`
Historique	`git log --oneline --graph --all`
Créer et changer de branche	`git switch -c <branche>` / `git switch <branche>`
Fusionner	`git merge <branche>`
Rebaser (branche perso)	`git rebase main`
Envoyer / récupérer	`git push` / `git fetch` / `git pull`
Mettre de côté	`git stash` / `git stash pop`

Besoin	Commande
Annuler un fichier modifié	`git restore <fichier>`
Annuler un commit poussé	`git revert <hash>`
Défaire un commit local	`git reset --soft HEAD~1`
Retrouver un commit perdu	`git reflog`
Taguer une version	`git tag -a v1.0.0 -m "..."` puis `git push origin v1.0.0`

⚠️ Les 6 erreurs les plus fréquentes

1. **Commiter un secret** (`.env` , clé API) → ajoute `.env` au `.gitignore` *avant* le premier commit ; si c'est trop tard, **change le secret** (il reste dans l'historique).
2. **Commiter des datasets lourds** → `.gitignore` , Git LFS ou DVC (GitHub refuse les fichiers > 100 Mo).
3. **Travailler directement sur** `main` → une branche par tâche.
4. **`git push --force` sur une branche partagée** → utiliser `--force-with-lease` , et seulement sur ta branche.
5. **`git reset --hard` sans réfléchir** → `git stash` d'abord, et `git reflog` pour réparer.
6. **Messages de commit vagues** (« update », « fix ») → dire *quoi* et *pourquoi*.

🔎 Note sur la fiabilité

Ce qui a été **réellement exécuté** pour vérification (Git 2.43, Python 3.13) : les tests et le lint du projet de démo, les étapes locales du TP (initialisation, ajout de l'écart-type et son test, conflit de merge, `restore` , `reset --soft` , `revert`), le filtrage du `.gitignore` , la validité YAML des 3 fichiers CI et la regex de couverture GitLab sur la vraie sortie de pytest. Les pipelines eux-mêmes n'ont **pas** été lancés sur GitHub/GitLab, et les étapes avec dépôt distant (push, PR/MR) n'ont pas pu être testées ici. Les éléments **qui évoluent vite** sont signalés dans le texte comme « à vérifier » : tarifs et quotas (minutes CI, offres GitKraken), versions majeures des actions GitHub (`@v4` , `@v5` …), libellés exacts des menus des interfaces graphiques. En cas d'écart, la documentation officielle fait foi :

- Git : https://git-scm.com/doc
- GitHub / Actions : https://docs.github.com
- GitHub Desktop : https://docs.github.com/desktop
- GitLab / CI/CD : https://docs.gitlab.com
- GitKraken : https://help.gitkraken.com

Chapitre 1 — Git : le moteur de tout le reste

> **À retenir d'abord :** Git est le **logiciel** de gestion de versions. GitHub, GitLab, GitHub Desktop, GitKraken et Git Bash sont des **outils autour de Git**. Si tu comprends Git, tous les autres deviennent faciles.

1.1 C'est quoi Git ?

Git est un **système de contrôle de version distribué** créé en 2005 par Linus Torvalds (pour le noyau Linux).

- **Contrôle de version** : il enregistre l'historique de tes fichiers. Tu peux revenir à n'importe quelle version, voir qui a changé quoi et pourquoi.
- **Distribué** : chaque copie du projet (sur ton PC, celui d'un collègue, le serveur) contient **tout l'historique**. Tu peux travailler hors ligne.

Analogie data scientist : Git, c'est comme si chaque version de ton notebook `analyse_v2_final_VRAI_final.ipynb` était sauvegardée proprement avec un message, une date, un auteur — sans dupliquer les fichiers.

1.2 Les 3 zones (le concept le plus important)



Zone	Rôle	Commande pour y mettre un fichier
Working directory	Tes fichiers tels que tu les édites	(tu modifies le fichier)
Staging area / index	« Panier » de ce qui ira dans le prochain commit	`git add`
Dépôt local	Historique des commits, dans le dossier caché `.git/`	`git commit`
Dépôt distant (remote)	Copie partagée sur un serveur	`git push`

Pourquoi la staging area ? Elle permet de faire des commits **ciblés** : tu as modifié 5 fichiers mais tu ne veux commiter que les 2 liés au nettoyage des données ? Tu n'ajoutes que ces 2-là.

1.3 Installation

Système	Méthode
Windows	Télécharger **Git for Windows** sur https://git-scm.com (installe aussi **Git Bash**, voir chapitre 2)
macOS	`xcode-select --install` ou `brew install git`
Linux (Debian/Ubuntu)	`sudo apt install git`

Vérifier :

```
git --version
# git version 2.43.0   (n'importe quelle version >= 2.28 convient pour ce cours)
```

> ⚠️ Les commandes `git switch` et `git restore` existent depuis **Git 2.23**, et `git init -b` depuis **Git 2.28**. Sur une version plus ancienne, utilise `git checkout` à la place.

1.4 Configuration initiale (une seule fois par machine)

```
git config --global user.name "Ton Nom"
git config --global user.email "ton.email@exemple.com"   # même email que sur GitHub/GitLab
git config --global init.defaultBranch main              # branche par défaut "main" au lieu de
"master"
git config --global core.editor "code --wait"            # VS Code comme éditeur (optionnel)
git config --global pull.rebase false                    # comportement de git pull = merge (explicite)

# Windows uniquement : gestion des fins de ligne
git config --global core.autocrlf true
# macOS / Linux :
git config --global core.autocrlf input

git config --list --show-origin   # voir toute la config et d'où elle vient
```

Niveaux de config : `--system` (toute la machine) < `--global` (ton utilisateur, fichier `~/.gitconfig`) < `--local` (le dépôt courant, `.git/config`). Le plus local gagne.

1.5 Créer ou récupérer un dépôt

```
# Option A : nouveau projet
mkdir mon-projet && cd mon-projet
git init                  # crée le dossier caché .git/

# Option B : récupérer un projet existant
git clone https://github.com/utilisateur/projet.git
git clone git@github.com:utilisateur/projet.git        # via SSH (voir chapitre 3)
git clone https://github.com/utilisateur/projet.git autre-nom   # dans un dossier nommé autrement
```

1.6 Le cycle de base : modifier → add → commit

```
echo "# Mon projet" > README.md
git status                        # README.md apparaît en "Untracked"
```

```
git add README.md               # ajoute au staging
git add .                       # ajoute TOUT le dossier courant (attention aux fichiers sensibles !)
git add -p                      # ajoute morceau par morceau, de façon interactive (très utile)

git commit -m "Ajoute le README"
git commit -am "Corrige une faute"   # add + commit des fichiers DÉJÀ suivis (pas les nouveaux)
```

Voir ce qui a changé

```
git status                # état global
git status -s             # version courte
git diff                  # différences working directory vs staging
git diff --staged         # différences staging vs dernier commit
git diff main..ma-branche  # différences entre deux branches
```

Lire l'historique

```
git log
git log --oneline                          # une ligne par commit
git log --oneline --graph --all            # arbre visuel de toutes les branches
git log -p fichier.py                      # historique détaillé d'un fichier
git log --author="Ton Nom" --since="2 weeks ago"
git show <hash>                            # détail d'un commit
git blame fichier.py                       # qui a écrit chaque ligne
```

Écrire un bon message de commit

```
Ajoute l'imputation par la médiane pour la colonne age     ← titre ≤ 50-72 caractères, à l'impératif

La moyenne était biaisée par des valeurs aberrantes (âge = 999).   ← corps : le POURQUOI
La médiane est plus robuste. Voir l'issue #12.
```

Convention répandue (**Conventional Commits**) : `feat: …`, `fix: …`, `docs: …`, `refactor: …`, `test: …`, `chore: …`.

1.7 Le fichier `.gitignore`

Liste les fichiers que Git doit **ignorer**. Indispensable en data science.

```
__pycache__/
.venv/
.ipynb_checkpoints/
.env              # secrets !
data/raw/         # données brutes lourdes
*.csv
!data/sample/*.csv   # le "!" ré-inclut une exception
models/*.pkl
```

Un modèle complet est fourni : `exemples/gitignore-data-science`.

⚠️ **Piège classique :** `.gitignore` n'agit **que sur les fichiers non encore suivis**. Si tu as déjà commité `data.csv`, l'ajouter au `.gitignore` ne suffit pas : `bash git rm --cached data.csv # arrête de suivre le fichier sans le supprimer du disque git commit -m "Arrête de suivre data.csv"` Et s'il contenait un **secret** (mot de passe, clé API), considère-le **compromis** : il reste dans l'historique. Change le secret immédiatement.

1.8 Les branches

Une branche est un **pointeur léger** vers un commit. Elle permet de développer une fonctionnalité sans casser `main`.

```
main:            A──B──C──────────F   (merge)
                          \         /
feature/nettoyage:         D──────E
```

```
git branch                          # lister les branches locales
git branch -a                       # + branches distantes
git switch -c feature/nettoyage     # créer ET basculer (ancien : git checkout -b feature/nettoyage)
git switch main                     # basculer (ancien : git checkout main)
git branch -d feature/nettoyage     # supprimer (refuse si non fusionnée)
git branch -D feature/nettoyage     # supprimer de force
git branch -m ancien-nom nouveau-nom
```

Fusionner : `merge`

```
git switch main
git merge feature/nettoyage
```

- **Fast-forward** : si `main` n'a pas bougé, Git avance simplement le pointeur (pas de commit de merge).
- **Merge commit** : si les deux branches ont divergé, Git crée un commit qui a deux parents.
- `git merge --no-ff feature/x` force un commit de merge (garde la trace de la branche).

Rebaser : `rebase`

`rebase` **rejoue** tes commits par-dessus une autre branche → historique linéaire.

```
Avant :  main: A──B──C          Après git rebase main (depuis feature) :
              \                  main:    A──B──C
     feature:  D──E                                \
                                    feature:        D'──E'
```

```
git switch feature/nettoyage
git rebase main
```

⚠️ **Règle d'or :** ne rebase **jamais** des commits déjà poussés sur une branche partagée par d'autres. Rebase réécrit l'historique (D devient D', nouveau hash). Sur ta branche perso, c'est OK — puis `git push --force-with-lease` (jamais `--force` tout court, qui peut écraser le travail d'un collègue).

	`merge`	`rebase`
Historique	Fidèle, avec commits de merge	Linéaire, plus lisible
Réécrit l'historique ?	Non	**Oui**
Sûr sur branche partagée ?	Oui	Non

1.9 Résoudre un conflit

Un conflit arrive quand deux branches modifient **les mêmes lignes** du même fichier.

```
git merge feature/nettoyage
# CONFLICT (content): Merge conflict in nettoyage.py
# Automatic merge failed; fix conflicts and then commit the result.
```

Git marque le fichier :

```
<<<<<<< HEAD
df["age"] = df["age"].fillna(df["age"].mean())
=======
df["age"] = df["age"].fillna(df["age"].median())
>>>>>>> feature/nettoyage
```

Procédure :

1. Ouvre le fichier, choisis la bonne version (ou combine), **supprime les marqueurs** `<<<<<<<`, `=======`, `>>>>>>>`.
2. `git add nettoyage.py`
3. `git commit` (pour un merge) ou `git rebase --continue` (pour un rebase).
4. En cas de panique : `git merge --abort` ou `git rebase --abort` → retour à l'état d'avant.

> 💡 Les conflits dans les **notebooks Jupyter** (`.ipynb` = JSON) sont pénibles. Solutions : `nbstripout` (retire les sorties avant commit), `nbdime` (diff/merge adapté aux notebooks), ou garder la logique dans des fichiers `.py` importés par le notebook.

1.10 Travailler avec un dépôt distant

```
git remote -v                                  # lister les remotes
git remote add origin git@github.com:moi/projet.git
git push -u origin main        # premier push : -u lie la branche locale à origin/main
git push                       # pushs suivants

git fetch origin               # télécharge les nouveautés SANS modifier tes fichiers
git pull                       # = git fetch + git merge (ou rebase selon la config)
git pull --rebase              # fetch + rebase → évite les commits de merge inutiles

git push origin --delete feature/nettoyage     # supprimer une branche distante
```

`fetch` vs `pull` : `fetch` est sans risque (il ne touche pas ta branche). `pull` fusionne immédiatement. En cas de doute : `git fetch` puis `git log --oneline main..origin/main` pour voir ce qui arrive.

1.11 Annuler des choses (tableau de survie)

Situation	Commande	Dangereux ?
Annuler les modifs d'un fichier non stagé	`git restore fichier.py`	⚠️ modifs perdues
Retirer un fichier du staging (garder les modifs)	`git restore --staged fichier.py`	Non
Modifier le message du dernier commit (non poussé)	`git commit --amend -m "Nouveau message"`	Réécrit l'historique
Ajouter un oubli au dernier commit (non poussé)	`git add oubli.py && git commit --amend --no-edit`	Réécrit l'historique

Situation	Commande	Dangereux ?
Défaire le dernier commit, garder les modifs stagées	`git reset --soft HEAD~1`	Réécrit l'historique
Défaire le dernier commit, garder les modifs non stagées	`git reset HEAD~1` (`--mixed` par défaut)	Réécrit l'historique
Défaire le dernier commit ET jeter les modifs	`git reset --hard HEAD~1`	⚠️⚠️
Annuler un commit **déjà poussé**	`git revert <hash>` (crée un commit inverse)	**Non — méthode sûre**
Retrouver un commit « perdu » après un reset	`git reflog` puis `git reset --hard <hash>`	Sauveur

💡 `git reflog` est ton filet de sécurité : il garde la trace de tous les déplacements de `HEAD`, même après un `reset --hard`. Par défaut, les entrées pointant vers des commits devenus inaccessibles (ex. après un `reset --hard`) sont conservées **30 jours** (`gc.reflogExpireUnreachable`), les autres 90 jours (`gc.reflogExpire`). N'attends donc pas pour récupérer un commit perdu.

1.12 Mettre de côté : `stash`

Tu es en plein travail et tu dois changer de branche en urgence :

```
git stash push -m "WIP feature engineering"   # met de côté les modifs (fichiers suivis)
git stash push -u -m "avec les nouveaux fichiers"   # -u inclut les fichiers non suivis
git stash list
git switch main   # ... tu corriges le bug urgent ...
git switch feature/x
git stash pop     # ré-applique et supprime le stash (git stash apply le garde)
```

1.13 Tags et versions

```
git tag -a v1.0.0 -m "Première version du modèle"   # tag annoté (recommandé)
git tag                                              # lister
git push origin v1.0.0                               # les tags ne sont PAS poussés par défaut
git push origin --tags
```

Convention **SemVer** : `MAJEUR.MINEUR.CORRECTIF` (ex. `2.1.3`).

1.14 Commandes avancées utiles

```
git cherry-pick <hash>        # copier UN commit d'une autre branche sur la branche courante
git rebase -i HEAD~3          # réorganiser/fusionner (squash)/renommer les 3 derniers commits
git bisect start              # recherche dichotomique du commit qui a introduit un bug
git bisect bad                #   le commit actuel est mauvais
git bisect good v1.0.0        #   celui-ci était bon → Git te fait tester le milieu, etc.
git bisect reset
git clean -n                  # liste les fichiers non suivis qui seraient supprimés
git clean -fd                 # ⚠️ les supprime vraiment
git worktree add ../hotfix main   # 2e dossier de travail sur une autre branche, même dépôt
```

1.15 Fichiers lourds : Git LFS et DVC

Git n'est pas fait pour les gros fichiers binaires (datasets, modèles). GitHub **refuse les fichiers de plus de 100 Mo** et avertit dès 50 Mo.

- **Git LFS** (Large File Storage) : remplace le fichier par un pointeur, le contenu est stocké à part. `bash git lfs install git lfs track "*.parquet" git add .gitattributes` Attention : quotas de stockage/bande passante LFS limités sur les offres gratuites.
- **DVC** (Data Version Control) : versionne données et modèles dans S3/GCS/Azure/Drive, Git ne garde que de petits fichiers `.dvc`. Très adapté aux pipelines ML.

1.16 Workflows d'équipe (stratégies de branches)

Stratégie	Principe	Pour qui
GitHub Flow	`main` toujours déployable ; une branche par fonctionnalité → Pull Request → merge	La plupart des équipes, projets data
GitLab Flow	GitHub Flow + branches d'environnement (`staging` , `production`)	Déploiements par environnement
Git Flow	`main` + `develop` + `feature/*` + `release/*` + `hotfix/*`	Logiciels à versions planifiées (plus lourd)
Trunk-based	Branches très courtes (< 1-2 jours), merge fréquent dans `main`	Équipes matures avec bonne CI

✅ Récapitulatif

1. `git add` → staging, `git commit` → historique local, `git push` → serveur.
2. Une branche par tâche, `main` reste propre.
3. `merge` est sûr ; `rebase` uniquement sur tes branches perso.
4. Déjà poussé ? → `git revert` , pas `git reset` .
5. Jamais de secrets ni de gros datasets dans Git.

📝 Exercices

Exercice 1.1 — Les 3 zones

Tu exécutes, dans un dépôt propre :

```
echo "v1" > a.py
git add a.py
echo "v2" > a.py
git status
```

a) Dans quelle(s) rubrique(s) `git status` affiche-t-il `a.py` ? b) Si tu fais maintenant `git commit -m "ajout"` , quel contenu de `a.py` est enregistré : `v1` ou `v2` ?

Exercice 1.2 — Premier historique

Écris la suite de commandes qui : crée un dossier `exo-git`, l'initialise avec la branche `main`, crée un `README.md` puis un `analyse.py`, et produit **deux commits séparés** (un par fichier). Termine par la commande qui affiche l'historique sur une ligne par commit.

Exercice 1.3 — Quelle commande ?

Pour chaque situation, donne **la** commande adaptée.

#	Situation
a	Tu as cassé `nettoyage.py` (non stagé) et veux revenir à la version du dernier commit
b	Tu as fait `git add data.csv` par erreur, mais tu veux garder le fichier sur le disque et ne pas le commiter
c	Ton dernier commit (pas encore poussé) a une faute dans le message
d	Un commit poussé il y a 3 jours sur `main` a introduit un bug ; tes collègues ont déjà récupéré `main`
e	Tu veux défaire ton dernier commit local mais garder les modifications stagées pour les recommiter autrement

Exercice 1.4 — Secret commité 🔐

Tu viens de faire un commit (**pas encore poussé**) qui contient, en plus de ton code, le fichier `.env` avec un mot de passe. Donne les commandes pour retirer `.env` de ce commit, l'ignorer à l'avenir, et garder le fichier sur ton disque. Que faudrait-il faire en plus si le commit avait **déjà été poussé** ?

Exercice 1.5 — Fast-forward ou commit de merge ?

Pour chaque cas, `git switch main && git merge feature` produit-il un fast-forward ou un commit de merge ?

```
Cas A :  main: A──B──C                Cas B :  main: A──B──C──F
                      \                                   \
               feature:  D──E                      feature:  D──E
```

Exercice 1.6 — Conflit pendant un rebase

Sur `main`, la ligne `seuil = 0.5` de `config.py` a été changée en `seuil = 0.7`. Sur ta branche `feature/modele`, tu l'as changée en `seuil = 0.6`. Tu lances `git rebase main` depuis ta branche → conflit. Tu décides de garder `0.7`. Quelles commandes tapes-tu pour terminer le rebase ? Et pour tout annuler ?

Exercice 1.7 — Sauvetage

Tu as tapé `git reset --hard HEAD~2` et perdu deux commits importants. Comment les récupérer ?

Exercice 1.8 — `.gitignore` piégé

Avec ce `.gitignore` :

```
*.csv
!data/sample/*.csv
data/raw/
!data/raw/LISEZMOI.md
```

Quels fichiers sont **ignorés** ? `ventes.csv`, `data/sample/extrait.csv`, `data/raw/clients.parquet`, `data/raw/LISEZMOI.md`.

🔑 Corrigés

Corrigé 1.1

a) **Dans les deux** : sous *Changes to be committed* (la version `v1` mise dans le staging) **et** sous *Changes not staged for commit* (la modification `v2` faite après le `add`). b) **`v1`** : `git commit` enregistre le contenu du staging, pas celui du disque. Pour enregistrer `v2`, il faut refaire `git add a.py` (ou `git commit -a`).

Corrigé 1.2

```
mkdir exo-git && cd exo-git
git init -b main
echo "# Exo Git" > README.md
git add README.md
git commit -m "docs: ajoute le README"
echo 'print("analyse")' > analyse.py
git add analyse.py
git commit -m "feat: ajoute le script d'analyse"
git log --oneline
```

Corrigé 1.3

#	Commande	Pourquoi
a	`git restore nettoyage.py`	Remet la version du dernier commit (⚠️ modifs perdues)
b	`git restore --staged data.csv`	Retire du staging, le fichier reste intact (et ajoute `*.csv` au `.gitignore`)
c	`git commit --amend -m "Nouveau message"`	Réécrit le dernier commit — OK car non poussé
d	`git revert <hash>`	Crée un commit inverse **sans réécrire** l'historique partagé
e	`git reset --soft HEAD~1`	Le commit disparaît, les modifs restent stagées

Corrigé 1.4

```
git rm --cached .env                # retire .env du suivi, sans le supprimer du disque
echo ".env" >> .gitignore
git add .gitignore
git commit --amend --no-edit        # remplace le commit fautif par une version sans .env
git log --stat -1                   # vérifier : .env n'apparaît plus
```

Si le commit **avait été poussé** : le mot de passe doit être considéré comme **compromis** → le **changer immédiatement**. Nettoyer l'historique distant (outil `git filter-repo`, puis force-push coordonné avec l'équipe) est possible mais ne suffit pas : des copies (clones, forks, caches) peuvent déjà exister.

Corrigé 1.5

- **Cas A : fast-forward.** `main` (C) est un ancêtre direct de `feature` (E) : Git avance simplement `main` jusqu'à E, aucun nouveau commit.
- **Cas B : commit de merge.** Les deux branches ont divergé depuis B (F d'un côté, D-E de l'autre) : Git crée un commit avec **deux parents** (F et E).

Corrigé 1.6

```
# Ouvrir config.py, garder "seuil = 0.7", supprimer les marqueurs <<<<<<< ======= >>>>>>>
git add config.py
```

```
git rebase --continue     # Git peut ouvrir l'éditeur pour confirmer le message : enregistrer et fermer
```

Pour tout annuler et revenir à l'état d'avant le rebase : `git rebase --abort`.

> Remarque : pendant un rebase, `HEAD` (« ours ») désigne la branche **sur laquelle** tu rejoues (`main`), et « theirs » tes propres commits — l'inverse d'un merge. C'est une source classique de confusion.

Corrigé 1.7

```
git reflog                    # repérer la ligne juste AVANT le "reset: moving to HEAD~2"
# ex. : a1b2c3d HEAD@{1}: commit: mon commit important
git reset --hard a1b2c3d      # ou : git reset --hard HEAD@{1}
```

Alternative plus prudente : `git branch sauvetage a1b2c3d` (crée une branche sur le commit perdu sans toucher à la branche actuelle).

Corrigé 1.8

Fichier	Ignoré ?	Raison
`ventes.csv`	✅ oui	règle `*.csv`
`data/sample/extrait.csv`	❌ non	ré-inclus par `!data/sample/*.csv`
`data/raw/clients.parquet`	✅ oui	tout le dossier `data/raw/` est ignoré
`data/raw/LISEZMOI.md`	✅ **oui** (piège !)	Git ne peut pas ré-inclure un fichier dont le **dossier parent** est exclu

Pour garder `LISEZMOI.md`, il faut ignorer le **contenu** du dossier et non le dossier lui-même :

```
data/raw/*
!data/raw/LISEZMOI.md
```

Vérification : `git check-ignore -v data/raw/LISEZMOI.md` affiche la règle responsable (aucune sortie = non ignoré).

Chapitre 2 — Git Bash : un terminal Linux sous Windows

2.1 C'est quoi ?

Git Bash est un terminal installé automatiquement avec **Git for Windows**. Il fournit :

- le shell **Bash** (le même que sous Linux/macOS) ;
- les commandes Unix courantes : `ls`, `cd`, `pwd`, `cat`, `grep`, `find`, `touch`, `mkdir`, `rm`, `cp`, `mv`, `head`, `tail`, `less`, `nano` / `vim` ;
- `ssh` et `ssh-keygen` (pour les clés SSH GitHub/GitLab) ;
- bien sûr la commande `git`.

Techniquement, c'est un environnement basé sur **MSYS2** affiché dans le terminal **MinTTY**.

> **Pourquoi l'utiliser ?** Les tutoriels Git, les docs et les scripts CI sont écrits en Bash. Avec Git Bash, tu tapes exactement les mêmes commandes que sur un serveur Linux ou dans GitHub Actions / GitLab CI. Sous macOS/Linux, tu n'en as pas besoin : ton terminal fait déjà tout ça.

2.2 Installation et ouverture

1. Télécharger Git for Windows sur https://git-scm.com/download/win.
2. Pendant l'installation, choix recommandés : - Éditeur par défaut : **VS Code** (ou Notepad++) plutôt que Vim si tu débutes ; - Nom de la branche initiale : **main** ; - « Git from the command line and also from 3rd-party software » ; - Fins de ligne : « Checkout Windows-style, commit Unix-style » (= `core.autocrlf true`).
3. Ouvrir : menu Démarrer → **Git Bash**, ou clic droit dans un dossier → **Open Git Bash here** (sous Windows 11 : « Afficher d'autres options »).

Dans VS Code : `Ctrl + Maj + P` → « Terminal: Select Default Profile » → **Git Bash**.

2.3 Les chemins : la différence à connaître

Windows (PowerShell/cmd)	Git Bash
`C:\Users\Nim\Documents`	`/c/Users/Nim/Documents`
`\` séparateur	`/` séparateur
`%USERPROFILE%`	`~` ou `$HOME`

```
cd /c/Users/Nim/Documents/projets
cd ~                # dossier personnel
cd ..               # dossier parent
cd -                # dossier précédent
pwd                 # où suis-je ?
explorer .          # ouvre le dossier courant dans l'Explorateur Windows
code .              # ouvre le dossier dans VS Code
```

> 💡 Les chemins avec espaces doivent être entre guillemets : `cd "/c/Users/Nim/Mes Documents"`.

2.4 Commandes Bash essentielles

```
ls                  # lister
ls -la              # lister tout, y compris les fichiers cachés (.git, .gitignore)
mkdir -p projet/data/raw   # créer une arborescence
touch notes.md      # créer un fichier vide
cat fichier.txt     # afficher le contenu
head -n 5 data.csv  # 5 premières lignes (pratique pour inspecter un CSV)
tail -f app.log     # suivre un log en direct
wc -l data.csv      # nombre de lignes
grep -rn "TODO" src/        # chercher "TODO" récursivement avec numéros de ligne
find . -name "*.ipynb"      # trouver tous les notebooks
cp source.py copie.py
mv ancien.py nouveau.py
rm fichier.txt
rm -r dossier/      # ⚠️ suppression définitive, pas de corbeille
history             # commandes précédentes  (flèche ↑ aussi)
clear               # nettoyer l'écran (ou Ctrl + L)
```

Raccourcis : `Tab` = autocomplétion (fichiers, branches Git !), `Ctrl + C` = interrompre, `Ctrl + R` = rechercher dans l'historique. Copier/coller dans MinTTY : `Ctrl + Inser` / `Maj + Inser` (ou sélection = copie, clic droit = coller selon la config).

2.5 Exemple complet : premier projet en Git Bash

```
cd /c/Users/Nim/Documents
mkdir analyse-ventes && cd analyse-ventes
git init
echo "# Analyse des ventes" > README.md
printf '__pycache__/\n.venv/\n.env\ndata/raw/\n' > .gitignore
git add README.md .gitignore
git commit -m "Initialise le projet"
git log --oneline
```

2.6 Créer une clé SSH (pour GitHub ou GitLab)

```
ssh-keygen -t ed25519 -C "ton.email@exemple.com"
# Appuie sur Entrée pour l'emplacement par défaut (~/.ssh/id_ed25519)
# Choisis une passphrase (recommandé)

eval "$(ssh-agent -s)"          # démarre l'agent SSH
ssh-add ~/.ssh/id_ed25519       # ajoute la clé à l'agent

cat ~/.ssh/id_ed25519.pub       # clé PUBLIQUE à copier dans GitHub/GitLab
clip < ~/.ssh/id_ed25519.pub    # Windows : copie directement dans le presse-papiers
```

Puis colle-la dans **GitHub → Settings → SSH and GPG keys** ou **GitLab → Preferences → SSH Keys**, et teste :

```
ssh -T git@github.com     # "Hi <utilisateur>! You've successfully authenticated..."
ssh -T git@gitlab.com     # "Welcome to GitLab, @<utilisateur>!"
```

⚠️ Ne partage **jamais** le fichier sans `.pub` (`id_ed25519`) : c'est ta clé privée.

2.7 Personnaliser : alias et `.bashrc`

Alias Git (stockés dans `~/.gitconfig`) :

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.sw switch
git config --global alias.lg "log --oneline --graph --all --decorate"
# Ensuite : git lg
```

Alias Bash (dans `~/.bashrc` , rechargé avec `source ~/.bashrc`) :

```
alias gs='git status'
alias activate='source .venv/Scripts/activate'   # venv Python sous Windows/Git Bash
```

💡 Sous Git Bash, un environnement virtuel Python s'active avec `source .venv/Scripts/activate` (dossier `Scripts` sous Windows, `bin` sous Linux/macOS).

2.8 Pièges fréquents

Problème	Cause / Solution
`python` ouvre le Microsoft Store ou ne répond pas	Désactive les « alias d'exécution d'application » Windows, ou utilise `py` ; si un script interactif bloque, préfixe avec `winpty python`
`warning: LF will be replaced by CRLF`	Normal avec `core.autocrlf true` ; ajoute un `.gitattributes` avec `* text=auto` pour uniformiser l'équipe
`Permission denied (publickey)`	Clé SSH non ajoutée à GitHub/GitLab ou à l'agent (voir 2.6)
Bloqué dans un écran avec `:` ou `~`	Tu es dans `less` ou Vim : tape `q` (less) ou `Échap` puis `:q!` + Entrée (Vim)

📝 Exercices

Exercice 2.1 — Chemins

Tu veux aller dans le dossier Windows `C:\Users\Nim\Documents\Projet Data` . Quelle commande tapes-tu dans Git Bash ?

Exercice 2.2 — Inspecter un CSV sans Python

Le fichier `ventes.csv` a une ligne d'en-tête. Écris les commandes qui : a) affichent l'en-tête seul ; b) comptent le nombre de **lignes de données** (sans l'en-tête) ; c) affichent les lignes contenant `Paris` .

Exercice 2.3 — Retrouver ses notebooks

Liste tous les fichiers `.ipynb` du projet **sauf** les copies automatiques stockées dans les dossiers `.ipynb_checkpoints`.

Exercice 2.4 — Chasse aux TODO

Affiche, avec numéros de ligne, toutes les occurrences de `TODO` dans les fichiers `.py` **uniquement** du dossier courant et de ses sous-dossiers.

Exercice 2.5 — Coincé dans Vim

Tu as tapé `git commit` sans `-m` et un écran étrange (Vim) s'est ouvert. a) Comment écrire le message, enregistrer et quitter ? b) Comment quitter **sans** commiter ? c) Comment éviter que ça se reproduise ?

Exercice 2.6 — `Permission denied (publickey)`

`git push` échoue avec `git@github.com: Permission denied (publickey).` Donne 4 vérifications à faire, dans l'ordre, avec les commandes.

🔑 Corrigés

Corrigé 2.1

```
cd "/c/Users/Nim/Documents/Projet Data"
```

Lettre de lecteur en minuscule précédée de `/`, des `/` comme séparateurs, et des **guillemets** à cause de l'espace (ou `Projet\ Data`).

Corrigé 2.2

```
head -n 1 ventes.csv                # a) en-tête
tail -n +2 ventes.csv | wc -l       # b) "tail -n +2" = à partir de la ligne 2
grep "Paris" ventes.csv             # c) (grep -i pour ignorer la casse)
```

⚠️ `wc -l` compte les retours à la ligne : si la dernière ligne du fichier n'en a pas, elle n'est pas comptée. Sur un vrai projet, vérifie avec pandas (`len(df)`).

Corrigé 2.3

```
find . -name "*.ipynb" -not -path "*/.ipynb_checkpoints/*"
```

Corrigé 2.4

```
grep -rn --include="*.py" "TODO" .
```

`-r` récursif, `-n` numéros de ligne, `--include` filtre les fichiers.

Corrigé 2.5

a) Appuie sur `i` (mode insertion), écris le message, puis `Échap`, tape `:wq` et `Entrée`. b) `Échap`, puis `:q!` et `Entrée` : le message reste vide, Git affiche *Aborting commit due to empty commit message* et ne crée pas de commit. c) Changer d'éditeur : `git config --global core.editor "code --wait"` (VS Code) ou `"nano"` ; ou toujours utiliser `git commit -m "..."`.

Corrigé 2.6

```
ls ~/.ssh/                          # 1. une clé existe-t-elle ? (id_ed25519 et id_ed25519.pub)
                                    #    sinon : ssh-keygen -t ed25519 -C "email"
eval "$(ssh-agent -s)" && ssh-add -l   # 2. la clé est-elle chargée dans l'agent ?
ssh-add ~/.ssh/id_ed25519              #    sinon, l'ajouter
ssh -T git@github.com               # 3. GitHub reconnaît-il la clé ? Sinon, coller le contenu
                                    #    de id_ed25519.pub dans Settings → SSH and GPG keys
git remote -v                       # 4. l'URL du remote est-elle bien en SSH (git@github.com:...) ?
                                    #    et le dépôt existe-t-il avec ce nom exact ?
```

Chapitre 3 — GitHub : héberger et collaborer

3.1 C'est quoi ?

GitHub (propriété de Microsoft depuis 2018) est une **plateforme web qui héberge des dépôts Git** et ajoute tout ce qu'il faut pour collaborer :

Fonction	Rôle
Repositories	Stockage des dépôts (publics ou privés)
Pull Requests (PR)	Proposer, relire et fusionner des changements
Issues	Tickets : bugs, tâches, idées
Projects	Tableaux Kanban / roadmap reliés aux issues
Actions	CI/CD intégrée (chapitre 5)
GitHub Pages	Héberger un site statique gratuit (portfolio, doc)
Codespaces	Environnement de dev dans le navigateur
Discussions, Wiki	Documentation et échanges
Security	Dependabot (mises à jour de dépendances), secret scanning, code scanning

> **Git ≠ GitHub.** Git fonctionne sans GitHub. GitHub n'est qu'un « serveur distant » (remote) parmi d'autres, avec une interface web.

3.2 Créer un compte et un dépôt

1. Créer un compte sur https://github.com.
2. Activer l'**authentification à deux facteurs** (Settings → Password and authentication).
3. Bouton **New** (ou `+` → *New repository*) : - nom : `analyse-ventes` ; - Public / Private ; - cocher éventuellement *Add a README*, un `.gitignore` (modèle **Python**) et une licence.

Lier un projet local existant

```
cd analyse-ventes
git remote add origin git@github.com:TON_UTILISATEUR/analyse-ventes.git
git branch -M main
git push -u origin main
```

3.3 S'authentifier : HTTPS ou SSH

GitHub **n'accepte plus les mots de passe** de compte pour `git push` en HTTPS. Deux options :

Méthode	Comment
SSH (recommandé)	Clé SSH (voir chapitre 2.6), URL `git@github.com:user/repo.git`

Méthode	Comment
HTTPS + token	Settings → Developer settings → *Personal access tokens* (préférer les *fine-grained tokens*, limités à certains dépôts et avec date d'expiration). Le token sert de mot de passe. Git Credential Manager (installé avec Git for Windows) le mémorise.
GitHub CLI	`gh auth login` configure tout automatiquement

3.4 Le workflow Pull Request (GitHub Flow)

C'est **la** façon de travailler en équipe sur GitHub.

```
1. Créer une branche  →  2. Commits  →  3. Push  →  4. Ouvrir une PR
→  5. Revue de code + CI verte  →  6. Merge dans main  →  7. Supprimer la branche
```

Exemple concret

```
git switch main
git pull                                        # partir d'un main à jour
git switch -c feature/imputation-age

# ... modifications de nettoyage.py ...
git add nettoyage.py
git commit -m "feat: impute l'âge manquant par la médiane"
git push -u origin feature/imputation-age
```

GitHub affiche alors un bandeau « **Compare & pull request** ». Dans la PR :

- **Titre** clair + **description** : quoi, pourquoi, comment tester ;
- `Closes #12` dans la description → l'issue #12 se ferme automatiquement au merge ;
- **Reviewers** : demande une relecture ;
- les relecteurs commentent ligne par ligne, puis *Approve* / *Request changes* ;
- pour corriger : nouveaux commits sur la même branche + `git push` → la PR se met à jour toute seule.

Les 3 boutons de merge

Option	Résultat dans `main`
Create a merge commit	Tous les commits + un commit de merge
Squash and merge	Un seul commit propre (très populaire)
Rebase and merge	Les commits rejoués linéairement, sans commit de merge

Après le merge, en local :

```
git switch main
git pull
git branch -d feature/imputation-age
git fetch --prune     # nettoie les références aux branches distantes supprimées
```

3.5 Contribuer à un projet dont tu n'es pas membre : le Fork

```
# 1. Sur GitHub : bouton "Fork" → copie du dépôt sous TON compte
git clone git@github.com:TOI/projet.git
```

```
cd projet
git remote add upstream git@github.com:AUTEUR/projet.git   # le dépôt original

# 2. Rester à jour avec l'original
git fetch upstream
git switch main
git merge upstream/main

# 3. Travailler sur une branche, push sur TON fork, puis ouvrir une PR vers AUTEUR/projet
```

3.6 Issues, labels, milestones, Projects

- **Issue** : un ticket (`#12`). Mentionne quelqu'un avec `@pseudo` , un commit avec son hash, une autre issue avec `#numéro` .
- **Labels** : `bug` , `enhancement` , `data` , `good first issue` ...
- **Milestones** : regrouper des issues par version/sprint.
- **Projects** : tableau Kanban (*Todo / In progress / Done*) où les cartes sont des issues et PR.

Mots-clés de fermeture automatique dans une PR ou un commit : `close` , `closes` , `fix` , `fixes` , `resolve` , `resolves` (+ `#numéro`).

3.7 Protéger `main`

Settings → Branches → Add branch ruleset (ou *branch protection rule*) :

- exiger une Pull Request avant de merger ;
- exiger 1+ approbation ;
- exiger que les checks CI passent (chapitre 5) ;
- interdire les force-push.

Fichier **`CODEOWNERS`** (dans `.github/`) pour assigner automatiquement des relecteurs :

```
# .github/CODEOWNERS
/models/     @equipe-ml
*.sql        @alice
```

3.8 Fichiers spéciaux reconnus par GitHub

Fichier	Effet
`README.md`	Page d'accueil du dépôt
`LICENSE`	Licence affichée
`.github/pull_request_template.md`	Modèle de description de PR
`.github/ISSUE_TEMPLATE/`	Modèles d'issues
`.github/workflows/*.yml`	Workflows GitHub Actions
`.github/dependabot.yml`	Mises à jour automatiques des dépendances
`CONTRIBUTING.md`	Guide de contribution

Fichier	Effet
`<pseudo>/<pseudo>` (dépôt au nom de ton compte) + `README.md`	README affiché sur ton profil — idéal pour un portfolio data

3.9 GitHub CLI (`gh`) — GitHub depuis le terminal

```
gh auth login
gh repo create analyse-ventes --private --source=. --push   # crée le dépôt et pousse
gh repo clone utilisateur/projet
gh issue list
gh issue create --title "Bug : NaN dans la colonne prix" --body "..."
gh pr create --fill               # ouvre une PR avec le titre/description issus des commits
gh pr list
gh pr checkout 42                 # récupère localement la branche de la PR #42
gh pr merge 42 --squash --delete-branch
gh run list                       # derniers runs GitHub Actions
gh run watch                      # suivre un run en direct
```

3.10 Astuces de pro

- Touche `.` sur une page de dépôt → ouvre l'éditeur web (VS Code dans le navigateur, github.dev).
- Touche `t` → recherche rapide de fichier ; `y` → lien permanent vers la version exacte d'un fichier.
- Les notebooks `.ipynb` sont rendus directement dans GitHub.
- **GitHub Pages** : Settings → Pages → publier depuis une branche ou via Actions → site sur `https://<pseudo>.github.io/<repo>/` .

📝 Exercices

Exercice 3.1 — Remettre le GitHub Flow dans l'ordre

Remets ces étapes dans le bon ordre : (a) Merge dans `main` · (b) `git switch -c feature/x` · (c) Ouvrir la Pull Request · (d) `git pull` sur `main` · (e) Commits · (f) Revue + CI verte · (g) `git push -u origin feature/x` · (h) Supprimer la branche

Exercice 3.2 — Push refusé

`git push` en HTTPS renvoie : *Support for password authentication was removed*. Que se passe-t-il, et quelles sont tes 3 options ? Donne la commande pour faire passer le remote `origin` en SSH.

Exercice 3.3 — Fermer deux issues automatiquement

Ta PR corrige les issues #4 et #7. Que dois-tu écrire dans sa description pour que les **deux** se ferment au merge ?

Exercice 3.4 — Mettre à jour son fork

Tu as forké `AUTEUR/projet` il y a un mois. Écris les commandes pour récupérer les nouveautés de l'original dans **ta** branche `main` locale **et** sur ton fork GitHub (le remote `upstream` n'est pas encore configuré).

Exercice 3.5 — Quel bouton de merge ?

a) L'équipe veut **un seul commit par fonctionnalité** dans `main`. b) Tu veux conserver tous les commits et la trace visible de la branche. c) Tu veux un historique linéaire mais en gardant chaque commit.

Exercice 3.6 — CODEOWNERS

Avec ce fichier, qui est demandé en relecture pour `sql/ventes.sql` ? Et pour `src/app.py` ?

```
*        @equipe-data
/sql/    @bob
```

🔑 Corrigés

Corrigé 3.1

d → b → e → g → c → f → a → h : partir d'un `main` à jour, créer la branche, commiter, pousser, ouvrir la PR, relecture + CI, merger, supprimer la branche.

Corrigé 3.2

GitHub n'accepte plus le **mot de passe du compte** pour Git en HTTPS. Options : 1. **SSH** (recommandé) ; 2. **HTTPS + Personal Access Token** (le token remplace le mot de passe ; Git Credential Manager le mémorise) ; 3. **`gh auth login`** (GitHub CLI configure l'authentification).

```
git remote set-url origin git@github.com:TON_UTILISATEUR/projet.git
git remote -v      # vérifier
```

Corrigé 3.3

```
Closes #4
Closes #7
```

Le mot-clé doit précéder **chaque** numéro : `Closes #4, #7` ne fermerait que la #4. (`Fixes` , `Resolves` … fonctionnent aussi.) La fermeture n'a lieu que si la PR est mergée dans la **branche par défaut**.

Corrigé 3.4

```
git remote add upstream https://github.com/AUTEUR/projet.git
git fetch upstream
git switch main
git merge upstream/main      # fast-forward si tu n'as pas commité sur ton main
git push origin main         # met à jour ton fork sur GitHub
```

(Raccourci web : bouton **Sync fork** sur la page de ton fork.)

Corrigé 3.5

a) **Squash and merge** · b) **Create a merge commit** · c) **Rebase and merge**.

Corrigé 3.6

- `sql/ventes.sql` → **@bob uniquement** : dans CODEOWNERS, c'est la **dernière règle qui correspond** qui l'emporte (pas un cumul).
- `src/app.py` → **@equipe-data** (seule la règle `*` correspond).

Chapitre 4 — GitHub Desktop : Git sans ligne de commande

4.1 C'est quoi ?

GitHub Desktop est une application graphique **gratuite et open source** éditée par GitHub. Elle exécute les commandes Git pour toi via des boutons.

✅ Points forts	❌ Limites
Très simple, idéale pour débuter	Pas de rebase interactif complet, pas de `bisect`, pas de `reflog`
Intégration native avec GitHub (login, PR, forks)	Pensée d'abord pour GitHub (fonctionne avec GitLab en clonant par URL, mais sans intégration)
Visualisation claire des diffs, commit ligne par ligne	Graphe des branches peu détaillé (GitKraken est meilleur sur ce point)
Windows et macOS	**Pas de version Linux officielle** (un fork communautaire existe : `shiftkey/desktop`)

> 💡 Conseil : utilise GitHub Desktop pour **voir** ce que tu fais, et apprends **en parallèle** les commandes équivalentes (tableau 4.4). Le jour où l'interface ne suffit plus, tu sauras quoi taper.

4.2 Installation et connexion

1. Télécharger sur https://desktop.github.com.
2. **File → Options → Accounts** (Windows) / **GitHub Desktop → Settings → Accounts** (macOS) → *Sign in to GitHub.com*.
3. **Options → Git** : vérifier nom et email (ceux des commits).

4.3 Utilisation pas à pas

a) Récupérer un dépôt

- **File → Clone repository** → onglet *GitHub.com* (tes dépôts) ou *URL* (n'importe quel dépôt, y compris GitLab) → choisir le dossier local → **Clone**.
- Ou **File → New repository** pour en créer un, puis **Publish repository** pour l'envoyer sur GitHub (case *Keep this code private* au besoin).
- Ou **File → Add local repository** pour un dossier déjà initialisé avec Git.

b) Faire un commit

1. Modifie tes fichiers dans ton éditeur (bouton **Open in Visual Studio Code**).
2. Onglet **Changes** (à gauche) : chaque fichier modifié a une case à cocher = staging. - Clic sur un fichier → diff à droite. Tu peux cliquer sur des **lignes** du diff pour n'inclure que certaines lignes (équivalent de `git add -p`).
3. En bas à gauche : **Summary** (titre) + *Description* → **Commit to main** (ou à la branche courante).

c) Synchroniser

Le bouton en haut change de nom selon la situation :

Bouton affiché	Signification	Commande équivalente
Publish branch	La branche n'existe pas encore sur GitHub	`git push -u origin <branche>`
Push origin (↑ n)	Tu as n commits locaux à envoyer	`git push`
Pull origin (↓ n)	n commits distants à récupérer	`git pull`
Fetch origin	Vérifier s'il y a du nouveau	`git fetch`

d) Branches

- Menu **Current branch** → **New branch** → nom `feature/visualisation` → *Create branch*.
- Changer de branche : clic sur une autre branche dans la liste. Si tu as des modifs non commitées, Desktop propose de les **laisser sur la branche actuelle (stash)** ou de les **emporter**.
- Fusionner : se placer sur `main` → **Branch → Merge into current branch…** → choisir la branche.
- Mettre à jour une branche : **Branch → Update from main** (merge) ou **Branch → Rebase current branch…**.

e) Pull Request

- Après le push d'une branche : bouton **Create Pull Request** (ou `Ctrl+R` / `Cmd+R`) → ouvre la page de création de PR dans le navigateur.
- **Branch → Preview Pull Request** pour vérifier le diff avant.
- Les PR existantes sont visibles dans le menu *Current branch → Pull requests* ; tu peux en récupérer une localement pour la tester.

f) Historique et annulations

Onglet **History** : - clic droit sur un commit → **Revert changes in commit** (`git revert`) ; - **Undo** (sur le dernier commit, s'il n'est pas poussé) → défait le commit et remet les changements dans *Changes* (`git reset --soft HEAD~1`) ; - **Amend commit** sur le dernier commit (`git commit --amend`) ; - **Cherry-pick** : glisser-déposer un commit sur une autre branche ; - **Squash** : glisser un commit sur un autre, ou sélection multiple → clic droit → *Squash* ; - **Create tag**.

Changements en cours à éviter de perdre : **Branch → Stash all changes**, puis *Restore* dans l'onglet *Changes*.

g) Conflits

Lors d'un merge en conflit, Desktop liste les fichiers concernés et propose **Open in Visual Studio Code** (ou ton éditeur). VS Code affiche *Accept Current / Accept Incoming / Accept Both*. Une fois les marqueurs supprimés, revenir dans Desktop → **Continue merge**.

4.4 Correspondance boutons ↔ commandes

GitHub Desktop	Ligne de commande
Clone repository	`git clone <url>`
Cocher un fichier dans *Changes*	`git add <fichier>`

GitHub Desktop	Ligne de commande
Commit to *branche*	`git commit -m "..."`
Push origin	`git push`
Pull origin	`git pull`
Fetch origin	`git fetch`
New branch	`git switch -c <branche>`
Merge into current branch	`git merge <branche>`
Revert changes in commit	`git revert <hash>`
Undo (dernier commit)	`git reset --soft HEAD~1`
Stash all changes	`git stash -u`
Discard changes	`git restore <fichier>` ⚠️
Repository → Open in Command Prompt / Git Bash	ouvre un terminal dans le dépôt

---

📝 Exercices

Exercice 4.1 — Boutons ↔ commandes

Donne la commande Git équivalente : a) *Fetch origin* · b) *Push origin* · c) *Undo* (sur le dernier commit) · d) *Discard changes* sur un fichier · e) *Revert changes in commit* · f) cocher/décocher un fichier dans *Changes*.

Exercice 4.2 — Commit partiel

Tu as modifié 30 lignes de `nettoyage.py`, mais seules 10 concernent le bug que tu corriges. Comment ne commiter que ces 10 lignes avec GitHub Desktop ?

Exercice 4.3 — Oups, j'ai codé sur `main`

Tu as des modifications non commitées sur `main` et tu réalises qu'elles devraient être sur une nouvelle branche `feature/graphiques`. Que fais-tu dans GitHub Desktop ?

Exercice 4.4 — Undo ou Revert ?

a) Tu viens de commiter, tu n'as pas poussé, et tu as oublié un fichier. b) Le commit a été poussé hier et tes collègues l'ont récupéré ; il faut l'annuler.

Exercice 4.5 — Lire le bouton

Le bouton du haut affiche **Pull origin** avec `↓ 2` et `↑ 1`. Qu'est-ce que ça signifie ? Que se passe-t-il quand tu cliques ?

Exercice 4.6 — Conflit

Remets dans l'ordre la résolution d'un conflit dans GitHub Desktop : (a) *Continue merge* · (b) *Branch → Merge into current branch…* · (c) supprimer les marqueurs et choisir le bon code dans l'éditeur · (d) *Open in Visual Studio Code* sur le fichier en conflit · (e) *Push origin*

🔑 Corrigés

Corrigé 4.1

a) `git fetch` · b) `git push` · c) `git reset --soft HEAD~1` · d) `git restore <fichier>` (⚠️ irréversible) · e) `git revert <hash>` · f) `git add <fichier>` / `git restore --staged <fichier>`.

Corrigé 4.2

Dans l'onglet *Changes*, clique sur `nettoyage.py` : le diff s'affiche. **Clique sur les numéros de ligne** (ou les blocs) que tu ne veux **pas** inclure pour les désélectionner. Seules les lignes sélectionnées iront dans le commit ; les autres restent en modifications locales. Équivalent : `git add -p`.

Corrigé 4.3

Current branch → *New branch* → `feature/graphiques` → *Create branch*. Desktop demande quoi faire des modifications : choisir de les **emporter sur la nouvelle branche** (*Bring my changes to feature/graphiques*). Puis commit sur cette branche. `main` reste propre. Équivalent : `git switch -c feature/graphiques` (les modifs non commitées suivent automatiquement).

Corrigé 4.4

a) **Undo** (ou *Amend commit*) : le dernier commit n'est pas poussé, on peut le refaire. b) **Revert changes in commit** : crée un commit inverse sans réécrire l'historique partagé.

Corrigé 4.5

Ta branche locale a **2 commits de retard** (présents sur GitHub, pas chez toi) et **1 commit d'avance** (chez toi, pas sur GitHub) : les historiques ont divergé. Cliquer sur *Pull origin* récupère et intègre les 2 commits (un merge par défaut, ou un rebase si ta config Git `pull.rebase` le demande — un conflit est possible). Ensuite le bouton devient *Push origin* ↑ pour envoyer ton commit (ou les 2 si un commit de merge a été créé).

Corrigé 4.6

b → d → c → a → e.

Chapitre 5 — GitHub Actions : automatiser (CI/CD) sur GitHub

5.1 C'est quoi la CI/CD ?

- **CI — Intégration Continue** : à chaque push / PR, une machine **teste automatiquement** le code (tests, lint, qualité). Objectif : détecter les erreurs avant qu'elles arrivent dans `main`.
- **CD — Déploiement / Livraison Continue** : si tout est vert, le code est **automatiquement déployé** (API, dashboard, package, modèle).

GitHub Actions est le système de CI/CD intégré à GitHub. Il se configure avec des **fichiers YAML** placés dans `.github/workflows/`.

5.2 Vocabulaire

```
Workflow  (fichier .github/workflows/ci.yml)
 └── déclenché par des Events  (on: push, pull_request, schedule…)
      └── contient des Jobs  (s'exécutent en parallèle par défaut, chacun sur un Runner)
           └── contiennent des Steps  (exécutées dans l'ordre)
                ├── run:  une commande shell
                └── uses: une Action réutilisable (ex. actions/checkout)
```

Terme	Définition
Workflow	Un fichier YAML = un processus automatisé
Event	Ce qui déclenche le workflow
Job	Un ensemble d'étapes exécutées sur la même machine
Runner	La machine virtuelle qui exécute le job (`ubuntu-latest` , `windows-latest` , `macos-latest` , ou un runner auto-hébergé)
Step	Une étape : commande (`run`) ou action (`uses`)
Action	Brique réutilisable publiée sur le **GitHub Marketplace**
Artifact	Fichier produit par un job et conservé (rapport, modèle…)
Secret	Variable chiffrée (mot de passe, clé API)

5.3 Premier workflow : « Hello »

Crée `.github/workflows/hello.yml` :

```yaml
name: Hello

on: [push]

jobs:
  dire-bonjour:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Bonjour depuis GitHub Actions !"
      - run: echo "Branche = ${{ github.ref_name }}, auteur = ${{ github.actor }}"
```

```
git add .github/workflows/hello.yml
git commit -m "ci: premier workflow"
git push
```

Résultat : onglet **Actions** du dépôt → le run apparaît, clique dessus pour voir les logs de chaque step.

> ⚠️ **YAML est sensible à l'indentation** : uniquement des **espaces** (jamais de tabulations), et une indentation cohérente (2 espaces par niveau ici).

5.4 Exemple réel : CI d'un projet Python data

Fichier complet : `exemples/github-actions/ci.yml` — il teste le mini-projet `exemples/projet-demo`.

```yaml
name: CI Python

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:          # bouton "Run workflow" manuel

permissions:
  contents: read              # moindre privilège pour le token automatique

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true    # annule le run précédent si on repousse sur la même branche

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # récupère le code du dépôt
      - uses: actions/setup-python@v5      # installe Python
        with:
          python-version: "3.12"
      - run: pip install ruff
      - run: ruff check .

  test:
    needs: lint                            # attend que "lint" réussisse
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12", "3.13"]   # 3 jobs en parallèle
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip                               # met en cache les paquets pip
          cache-dependency-path: requirements-dev.txt
      - run: pip install -r requirements-dev.txt
      - run: python -m pytest --cov=demo --cov-report=term --cov-report=xml
      - if: matrix.python-version == '3.12'
        uses: actions/upload-artifact@v4           # garde le rapport téléchargeable
        with:
          name: coverage-report
          path: coverage.xml
```

Explications ligne à ligne :

- `on:` → déclencheurs : push sur `main`, toute PR ciblant `main`, et lancement manuel.

- `needs: lint` → crée une dépendance : `test` ne tourne pas si le lint échoue.
- `matrix` → GitHub crée **un job par version** de Python.
- `${{ ... }}` → **expression** évaluée par GitHub (variables, contextes, fonctions).
- `if:` → condition d'exécution d'un step ou d'un job.

> 💡 **Versions des actions :** `@v4`, `@v5` désignent des versions majeures. Des versions majeures plus récentes peuvent exister au moment où tu lis ce cours : vérifie sur la page Marketplace de chaque action. Pour plus de sécurité, les équipes exigeantes épinglent sur un **hash de commit** (`uses: actions/checkout@<sha>`).

Couplé à une **règle de protection de branche** « Require status checks to pass » (chapitre 3.7), une PR ne peut plus être mergée si les tests échouent.

5.5 Les déclencheurs (`on:`) les plus utiles

```yaml
on:
  push:
    branches: [main, "release/**"]
    paths: ["src/**", "tests/**"]      # seulement si ces fichiers changent
  pull_request:
  schedule:
    - cron: "0 6 * * 1"                # chaque lundi 06:00 UTC
  workflow_dispatch:                   # manuel, avec paramètres possibles :
    inputs:
      environnement:
        type: choice
        options: [staging, production]
  release:
    types: [published]
```

Format cron : `minute heure jour-du-mois mois jour-de-la-semaine`, **toujours en UTC**. Le démarrage d'un workflow planifié peut être retardé aux heures de forte charge.

5.6 Secrets et variables

1. **Settings → Secrets and variables → Actions → New repository secret** → nom `DB_URL`, valeur.
2. Utilisation :

```yaml
      - name: Générer le rapport
        env:
          DB_URL: ${{ secrets.DB_URL }}
        run: python scripts/rapport.py
```

- Les secrets sont **masqués** (`***`) dans les logs.
- Ils ne sont **pas transmis** aux workflows déclenchés par des PR venant de **forks** (sécurité).
- `vars.MA_VARIABLE` pour les variables non sensibles.
- `secrets.GITHUB_TOKEN` est créé automatiquement à chaque run ; ses droits sont réglés par `permissions:`.

Exemple data planifié complet : `exemples/github-actions/rapport-hebdo.yml`.

5.7 Déploiement continu (CD) : exemple GitHub Pages

Publier un dossier `site/` (ex. rapport HTML généré, doc MkDocs, Quarto) :

```yaml
name: Déployer le site

on:
  push:
    branches: [main]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: mkdir -p site && echo "<h1>Rapport</h1>" > site/index.html
      - uses: actions/upload-pages-artifact@v3
        with:
          path: site

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4
```

Prérequis : **Settings → Pages → Source : GitHub Actions**.

Environments (Settings → Environments) : tu peux exiger une **approbation manuelle** avant un déploiement en production et y stocker des secrets propres à chaque environnement.

5.8 Bonnes pratiques et coûts

- Dépôts **publics** : minutes gratuites illimitées sur les runners standards. Dépôts **privés** : quota mensuel de minutes selon l'offre (2 000 min/mois sur l'offre Free à ma connaissance — **vérifie la page de tarification GitHub**, ces quotas évoluent). Les runners macOS/Windows consomment le quota plus vite.
- Mets en **cache** les dépendances (`cache: pip`).
- Utilise `concurrency` pour éviter les runs inutiles.
- Donne le **minimum de permissions** (`permissions:`).
- Ne mets **jamais** un secret en clair dans le YAML.
- Méfie-toi des actions tierces peu connues : ce sont des dépendances qui exécutent du code avec accès à ton dépôt.

Déboguer un workflow

- Lire les logs dans l'onglet **Actions** (chaque step se déplie).
- **Re-run jobs → Enable debug logging** pour des logs détaillés.
- Tester en local avec l'outil open source **`act`** (exécute les workflows dans Docker — pas parfaitement identique à GitHub).

📝 Exercices

Exercice 5.1 — Trouve les 4 erreurs

Ce workflow ne fonctionne pas. Trouve les erreurs.

```yaml
name: CI
on: push
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses actions/checkout@v4
      - run: pip install ruff && ruff check .
  test:
    needs: build
    steps:
      - uses: actions/checkout@v4
      - run: python -m pytest
```

Exercice 5.2 — Déclencheur ciblé

Écris le bloc `on:` d'un workflow qui se lance **uniquement** sur les Pull Requests vers `main` **qui modifient** des fichiers dans `src/` ou `tests/`, ainsi qu'en lancement manuel.

Exercice 5.3 — Cron

Tu veux lancer un rapport **du lundi au vendredi à 7h30, heure de Paris**. Le cron de GitHub Actions est en UTC. Quelle expression écris-tu en hiver ? En été ? Quel problème cela pose-t-il ?

Exercice 5.4 — Combien de jobs ?

Combien de jobs cette matrice génère-t-elle ? Lesquels ?

```yaml
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    python-version: ["3.11", "3.12"]
    exclude:
      - os: windows-latest
        python-version: "3.11"
```

Exercice 5.5 — Déployer seulement depuis `main`

Ajoute à un workflow existant (qui a un job `test`) un job `deploy` qui : - ne démarre **qu'après la réussite** de `test` ; - ne s'exécute **que** sur un `push` vers `main` (pas sur les PR) ; - utilise le secret `DEPLOY_TOKEN` dans une variable d'environnement.

Exercice 5.6 — Secrets et forks

a) Pourquoi ne faut-il jamais écrire `run: echo ${{ secrets.API_KEY }}` pour « vérifier » un secret ? b) Un contributeur externe ouvre une PR depuis son fork : ton workflow échoue car `secrets.API_KEY` est vide. Bug ou comportement voulu ?

🔑 Corrigés

Corrigé 5.1

1. `- uses actions/checkout@v4` → il manque les **deux-points** : `- uses: actions/checkout@v4`.
2. `needs: build` → aucun job ne s'appelle `build` : `needs: lint`.
3. Le job `test` n'a **pas de** `runs-on:` (obligatoire).
4. `test` lance pytest **sans installer** Python ni les dépendances (pytest n'est pas garanti sur le runner) : ajouter `actions/setup-python` et `pip install -r requirements-dev.txt`.

Version corrigée :

```yaml
name: CI
on: push
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install ruff && ruff check .
  test:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements-dev.txt
      - run: python -m pytest
```

💡 L'outil `actionlint` détecte automatiquement ce type d'erreurs avant le push.

Corrigé 5.2

```yaml
on:
  pull_request:
    branches: [main]
    paths:
      - "src/**"
      - "tests/**"
  workflow_dispatch:
```

Corrigé 5.3

Paris = UTC+1 en hiver, UTC+2 en été (heure d'été). - Hiver : `cron: "30 6 * * 1-5"` (6h30 UTC = 7h30 Paris) - Été : `cron: "30 5 * * 1-5"` (5h30 UTC = 7h30 Paris)

Problème : avec un cron fixe en UTC, l'heure locale **se décale d'une heure** deux fois par an. Solutions : accepter le décalage, ou mettre les deux crons et faire vérifier l'heure locale par le script (`TZ=Europe/Paris date +%H`). Rappel : le démarrage d'un workflow planifié peut aussi être retardé de plusieurs minutes.

Corrigé 5.4

3 jobs : 2 OS × 2 versions = 4, moins la combinaison exclue (windows + 3.11) → `ubuntu/3.11` , `ubuntu/3.12` , `windows/3.12` .

Corrigé 5.5

```yaml
deploy:
  needs: test
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Déployer
      env:
        DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
      run: ./scripts/deploy.sh
```

`github.ref` vaut `refs/heads/<branche>` pour un push (on peut aussi écrire `github.ref_name == 'main'`).

Corrigé 5.6

a) GitHub masque les secrets connus (`***`) dans les logs, mais ce masquage n'est pas infaillible (valeur transformée, encodée, découpée…). Afficher un secret, c'est risquer de le divulguer dans des logs souvent lisibles par beaucoup de monde. Pour vérifier qu'il est défini : `if [ -z "$API_KEY" ]; then echo "absent"; exit 1; fi`. b) **Comportement voulu** : GitHub ne transmet pas les secrets aux workflows déclenchés par des PR venant de forks, sinon n'importe qui pourrait les voler en modifiant le workflow dans sa PR. Il faut que les jobs qui en ont besoin soient sautés dans ce cas, ou s'exécutent après le merge.

Chapitre 6 — GitLab : la plateforme DevOps tout-en-un

6.1 C'est quoi ?

GitLab est, comme GitHub, une **plateforme d'hébergement de dépôts Git** avec outils de collaboration. Sa particularité : elle se veut une plateforme **DevOps complète** (code, CI/CD, registre de conteneurs, sécurité, déploiement, suivi) **et** elle peut être **installée sur tes propres serveurs**.

Deux façons de l'utiliser :

	GitLab.com (SaaS)	GitLab Self-Managed
Hébergement	Par GitLab Inc.	Sur les serveurs de ton entreprise
Usage typique	Projets perso, startups	Banques, administrations, industries (données sensibles, souveraineté)
Éditions	Free / Premium / Ultimate	Community Edition (open source, gratuite) / Enterprise Edition

> 💡 En entreprise (surtout en France et en Europe), tu croiseras souvent un GitLab **interne** (ex. `gitlab.monentreprise.fr`). Les commandes Git sont identiques ; seule l'URL change.

6.2 GitHub vs GitLab : le dictionnaire

GitHub	GitLab	Remarque
Repository	**Project**	Même chose
Organization	**Group** (+ sous-groupes)	GitLab permet des **sous-groupes imbriqués**
Pull Request (PR)	**Merge Request (MR)**	Même concept
GitHub Actions	**GitLab CI/CD**	Fichier unique `.gitlab-ci.yml` (chapitre 7)
Workflow	**Pipeline**	
Runner	**Runner**	`gitlab-runner` pour les runners auto-hébergés
Actions Marketplace	**CI/CD Catalog**, templates, `include:`	
GitHub Pages	**GitLab Pages**	
GitHub Packages / ghcr.io	**Package Registry** / **Container Registry**	
`gh` (CLI)	`glab` (CLI)	
Collaborator	**Member** avec un **rôle**	Rôles : Guest, Reporter, Developer, Maintainer, Owner (+ Planner sur les versions récentes)

6.3 Créer un compte, un projet, et pousser

1. Compte sur https://gitlab.com (activer la 2FA).
2. Ajouter ta clé SSH : avatar → **Preferences → SSH Keys** (voir chapitre 2.6).
3. **New project → Create blank project** → nom, *visibility* (Private / Internal / Public).

> *Internal* = visible par tous les utilisateurs connectés de l'instance (utile sur un GitLab d'entreprise).

Pousser un projet existant

```
cd analyse-ventes
git remote add origin git@gitlab.com:TON_UTILISATEUR/analyse-ventes.git
git push -u origin main
```

Astuce : créer le projet en poussant

GitLab peut créer un projet **privé** directement au premier push :

```
git push --set-upstream git@gitlab.com:TON_UTILISATEUR/nouveau-projet.git main
```

Avoir GitHub ET GitLab en même temps

```
git remote add github git@github.com:moi/analyse-ventes.git
git remote add gitlab git@gitlab.com:moi/analyse-ventes.git
git push github main
git push gitlab main
```

(GitLab propose aussi le **mirroring** automatique : Settings → Repository → Mirroring repositories.)

6.4 Le workflow Merge Request

Identique au workflow PR de GitHub :

```
git switch -c feature/dashboard
# ... travail ...
git commit -am "feat: ajoute le dashboard des ventes"
git push -u origin feature/dashboard
```

Git renvoie directement un lien :

```
remote: To create a merge request for feature/dashboard, visit:
remote:   https://gitlab.com/moi/analyse-ventes/-/merge_requests/new?
merge_request%5Bsource_branch%5D=feature%2Fdashboard
```

Push options (spécifiques à GitLab) — créer la MR sans ouvrir le navigateur :

```
git push -o merge_request.create \
         -o merge_request.target=main \
         -o merge_request.title="Ajoute le dashboard" \
         -o merge_request.remove_source_branch
```

Dans la MR : - **Draft** : préfixe le titre par `Draft:` tant que ce n'est pas prêt ; - **Reviewers / Assignees**, **Approvals** (nombre d'approbations requises : fonctionnalité avancée selon l'offre) ; - **Pipeline** : le résultat de GitLab CI s'affiche dans la MR ; - option *Merge when pipeline succeeds* (« Auto-merge » sur les

versions récentes) ; - options *Squash commits* et *Delete source branch*.

Quick actions dans les commentaires d'issues/MR :

```
/assign @alice
/label ~bug ~data
/milestone %v1.0
/estimate 2h
/spend 30m
/close
```

Référencer : `#12` (issue), `!34` (merge request), `~bug` (label), `%v1.0` (milestone), `@alice` (utilisateur).

6.5 Issues et planification

- **Issues** avec labels, poids, dates d'échéance, estimation et temps passé (`/estimate` , `/spend`).
- **Issue boards** : tableaux Kanban dont les colonnes sont des labels.
- **Milestones** et **Iterations** (sprints).
- Fermeture automatique : `Closes #12` dans une MR ou un commit.

6.6 Protéger les branches

Settings → Repository → Protected branches : - *Allowed to merge* : Maintainers ; - *Allowed to push and merge* : No one → oblige à passer par une MR ; - interdire le force push.

Settings → Merge requests : *Pipelines must succeed*, *All threads must be resolved*.

Fichier `CODEOWNERS` (à la racine, dans `docs/` ou `.gitlab/`) : même principe que sur GitHub, mais sur GitLab c'est une fonctionnalité des offres **Premium/Ultimate** (pas disponible en Free).

6.7 GitLab CLI (`glab`)

```
glab auth login
glab repo clone groupe/projet
glab issue list
glab issue create --title "Bug : doublons dans clients" --label bug
glab mr create --fill --target-branch main
glab mr list
glab mr checkout 34
glab mr merge 34 --squash
glab ci status          # état du pipeline de la branche courante
glab ci view            # vue interactive du pipeline
```

6.8 Autres fonctionnalités notables

- **Web IDE** : touche `.` sur un projet, ou bouton *Edit → Web IDE*.
- **Container Registry** : stocker les images Docker de tes modèles/API (`registry.gitlab.com/groupe/projet`).
- **Package Registry** : publier des paquets Python (PyPI privé), npm, Maven…
- **Environments & Deployments** : suivi de ce qui est déployé où.

- **Model registry / Experiments** (MLflow-compatible) : selon la version et l'offre — à vérifier sur ton instance.
- **Security scanning** (SAST, détection de secrets, dépendances) via des templates CI ; certaines fonctionnalités sont réservées à l'offre Ultimate.

📝 Exercices

Exercice 6.1 — Traduire GitHub → GitLab

Donne l'équivalent GitLab de : Pull Request · Organization · Repository · GitHub Actions · `gh` · Workflow.

Exercice 6.2 — Références

Dans un commentaire GitLab, que désignent : `#12`, `!34`, `~bug`, `%v1.0`, `@alice` ?

Exercice 6.3 — Quick actions

Écris un commentaire qui, sur une issue : t'assigne l'issue, ajoute les labels `bug` et `urgent`, la rattache au milestone `v2.0` et estime le travail à 3 heures.

Exercice 6.4 — MR en une commande

Tu es sur la branche `feature/export` (déjà commitée). Pousse-la en créant directement une Merge Request **brouillon** vers `develop`, avec le titre « Export Parquet » et suppression de la branche source après merge.

Exercice 6.5 — Le bon rôle

Quel rôle minimal donner à chacun (Guest, Reporter, Developer, Maintainer, Owner) ? a) Une stagiaire qui doit lire le code et créer des issues, sans pousser de code. b) Un data scientist qui pousse sur des branches de fonctionnalité et ouvre des MR, sans pouvoir pousser sur `main` protégée. c) Le lead qui configure les branches protégées et les variables CI/CD.

Exercice 6.6 — Pousser vers GitHub ET GitLab d'un coup

Configure le remote `origin` pour qu'un seul `git push` envoie sur GitHub **et** GitLab. Attention au piège !

🔑 Corrigés

Corrigé 6.1

Merge Request · Group · Project · GitLab CI/CD · `glab` · Pipeline.

Corrigé 6.2

`#12` issue n°12 · `!34` merge request n°34 · `~bug` label « bug » · `%v1.0` milestone « v1.0 » · `@alice` mention de l'utilisatrice alice (elle est notifiée).

Corrigé 6.3

```
/assign me
/label ~bug ~urgent
/milestone %v2.0
/estimate 3h
```

Corrigé 6.4

```
git push -u origin feature/export \
  -o merge_request.create \
  -o merge_request.target=develop \
  -o merge_request.title="Draft: Export Parquet" \
  -o merge_request.remove_source_branch
```

Le préfixe `Draft:` dans le titre marque la MR comme brouillon (elle ne peut pas être mergée tant qu'il n'est pas retiré).

Corrigé 6.5

a) **Reporter** (lecture du code + issues ; un Guest ne voit pas le code d'un projet privé). b) **Developer** (push sur les branches non protégées, création de MR). c) **Maintainer** (paramètres du projet, branches protégées, variables CI/CD).

Corrigé 6.6

```
git remote set-url --add --push origin git@github.com:moi/projet.git
git remote set-url --add --push origin git@gitlab.com:moi/projet.git
git remote -v
# origin  git@github.com:moi/projet.git (fetch)
# origin  git@github.com:moi/projet.git (push)
# origin  git@gitlab.com:moi/projet.git (push)
```

Le piège : dès qu'on ajoute une URL de push explicite, elle **remplace** l'URL par défaut. Si tu n'ajoutes que GitLab, `git push` n'irait plus que sur GitLab : il faut donc ajouter **les deux**. Le `fetch` continue de lire une seule source (ici GitHub).

Chapitre 7 — GitLab CI/CD : pipelines d'automatisation

7.1 Principe

GitLab CI/CD fait le même travail que GitHub Actions (chapitre 5), avec une différence d'organisation :

- **un seul fichier** à la racine du projet : `.gitlab-ci.yml` (on peut l'éclater avec `include:`) ;
- le pipeline est découpé en **stages** (étapes) exécutés **dans l'ordre** ; les **jobs** d'un même stage tournent **en parallèle** ;
- les jobs sont exécutés par des **GitLab Runners** (partagés sur GitLab.com, ou installés par toi), le plus souvent **dans un conteneur Docker** défini par `image:`.

```
Pipeline
 ├── stage: lint     →  [lint]
 ├── stage: test     →  [test 3.11] [test 3.12] [test 3.13]   (parallèles)
 └── stage: deploy   →  [deploy_staging]  [deploy_production ▶ manuel]
```

Si un job d'un stage échoue, les stages suivants ne démarrent pas (par défaut).

7.2 Premier pipeline

Crée `.gitlab-ci.yml` à la racine :

```yaml
bonjour:
  image: alpine:latest
  script:
    - echo "Bonjour depuis GitLab CI !"
    - echo "Branche = $CI_COMMIT_BRANCH, commit = $CI_COMMIT_SHORT_SHA"
```

```
git add .gitlab-ci.yml
git commit -m "ci: premier pipeline"
git push
```

Résultat : **Build → Pipelines** (menu de gauche) → clique sur le pipeline puis sur le job pour voir les logs.

> 💡 Les stages par défaut sont `.pre`, `build`, `test`, `deploy`, `.post`. Un job sans `stage:` est dans `test`.
>
> 💡 **Valider avant de pousser : Build → Pipeline editor** vérifie la syntaxe et affiche une visualisation du pipeline.

7.3 Exemple réel : projet Python data

Fichier complet : `exemples/gitlab-ci/.gitlab-ci.yml` — il teste le mini-projet `exemples/projet-demo`.

```yaml
stages:
  - lint
  - test
  - deploy
```

```yaml
default:
  image: python:3.12-slim          # image Docker utilisée par défaut par chaque job

variables:
  PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"

workflow:                          # QUAND créer un pipeline
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never                  # évite un 2e pipeline de branche si une MR est ouverte
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_TAG

cache:
  key:
    files:
      - requirements-dev.txt       # le cache change quand ce fichier change
  paths:
    - .cache/pip

lint:
  stage: lint
  script:
    - pip install ruff
    - ruff check .

test:
  stage: test
  parallel:
    matrix:
      - PYTHON_VERSION: ["3.11", "3.12", "3.13"]
  image: python:${PYTHON_VERSION}-slim
  script:
    - pip install -r requirements-dev.txt
    - python -m pytest --cov=demo --cov-report=term --cov-report=xml
  coverage: '/TOTAL.*\s+(\d+%)$/'  # extrait le % de couverture des logs → affiché dans la MR
  artifacts:
    when: always
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml         # couverture ligne par ligne dans le diff de la MR
    expire_in: 1 week

deploy_staging:
  stage: deploy
  script:
    - echo "Déploiement sur staging…"
  environment:
    name: staging
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

deploy_production:
  stage: deploy
  script:
    - echo "Déploiement en production de $CI_COMMIT_TAG"
  environment:
    name: production
  rules:
    - if: $CI_COMMIT_TAG
      when: manual                 # bouton ▶ : un humain valide la mise en production
```

Explications :

Mot-clé	Rôle
`stages`	Ordre des étapes
`default: image`	Conteneur Docker dans lequel tournent les jobs

Mot-clé	Rôle
`variables`	Variables d'environnement
`workflow: rules`	Décide si un pipeline est créé
`rules` (dans un job)	Décide si **ce job** est ajouté au pipeline (remplace les anciens `only/except`, dépréciés)
`script`	Commandes shell exécutées (obligatoire)
`before_script` / `after_script`	Commandes avant/après `script`
`parallel: matrix`	Plusieurs copies du job avec des variables différentes
`cache`	Accélère les jobs suivants (dépendances) — **pas garanti**
`artifacts`	Fichiers produits, conservés et transmis aux jobs des stages suivants
`coverage`	Regex qui lit le % de couverture dans les logs
`environment`	Trace les déploiements (menu **Operate → Environments**)
`when: manual`	Job lancé à la main
`needs`	Lance un job dès que ses dépendances sont finies, sans attendre tout le stage (pipeline en graphe, plus rapide)

> **Cache vs artifacts :** le *cache* sert à accélérer (dépendances téléchargées), il peut être absent. Les *artifacts* sont des **résultats** (rapport, modèle entraîné, build) que tu veux garder ou passer au job suivant : ils sont garantis.

7.4 Variables prédéfinies utiles

Variable	Contenu
`$CI_COMMIT_BRANCH`	Branche du commit (vide dans un pipeline de MR ou de tag)
`$CI_DEFAULT_BRANCH`	Branche par défaut (`main`)
`$CI_COMMIT_TAG`	Tag (si pipeline de tag)
`$CI_COMMIT_SHORT_SHA`	Hash court du commit
`$CI_PIPELINE_SOURCE`	Origine : `push`, `merge_request_event`, `schedule`, `web`…
`$CI_PROJECT_DIR`	Dossier où le code est cloné
`$CI_REGISTRY_IMAGE`	Adresse du Container Registry du projet
`$CI_JOB_TOKEN`	Jeton temporaire du job

7.5 Variables secrètes

Settings → CI/CD → Variables → Add variable :

- **Masked** (« Mask variable ») : la valeur est cachée dans les logs ;
- **Protected** : disponible uniquement sur les branches/tags protégés (ex. `main`) ;

- type *File* : GitLab écrit la valeur dans un fichier et la variable contient son chemin (pratique pour un fichier de credentials JSON).

```
entrainer:
  script:
    - python train.py --db "$DB_URL"      # DB_URL définie dans l'interface, jamais dans le YAML
```

7.6 Exemples data / ML

Pipeline planifié (ré-entraînement hebdomadaire)

1. **Build → Pipeline schedules → New schedule** : cron `0 6 * * 1`, fuseau horaire choisi, branche `main`.
2. Dans le YAML :

```
reentrainement:
  stage: deploy
  script:
    - pip install -r requirements.txt
    - python train.py --sortie models/model.joblib
  artifacts:
    paths:
      - models/model.joblib
    expire_in: 30 days
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
```

Construire et pousser une image Docker (API de prédiction)

```
build_image:
  stage: deploy
  image: docker:27
  services:
    - docker:27-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  script:
    - echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --password-stdin
"$CI_REGISTRY"
    - docker build -t "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA" .
    - docker push "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

> Docker-in-Docker (`dind`) nécessite un runner en mode privilégié : c'est le cas des runners partagés Linux de GitLab.com, mais pas forcément d'un runner d'entreprise. Alternatives sans privilèges : Kaniko (projet désormais archivé par Google, des forks existent) ou Buildah — à valider avec ton équipe infra.

GitLab Pages (publier un rapport HTML)

```
pages:
  stage: deploy
  script:
    - mkdir -p public
    - echo "<h1>Rapport des ventes</h1>" > public/index.html
  artifacts:
    paths:
      - public                    # GitLab publie ce dossier
```

```yaml
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

7.7 Réutiliser de la configuration : `include` et `extends`

```yaml
include:
  - local: ci/tests.yml                      # autre fichier du même projet
  - project: mon-groupe/ci-templates         # fichier d'un autre projet
    file: python.yml
  - template: Jobs/SAST.gitlab-ci.yml        # template officiel GitLab (analyse de sécurité)

.python-base:                # un job commençant par "." est caché : c'est un modèle
  image: python:3.12-slim
  before_script:
    - pip install -r requirements.txt

tests-unitaires:
  extends: .python-base
  script:
    - python -m pytest
```

7.8 Les runners

- **GitLab.com** fournit des runners partagés (Linux, et selon l'offre Windows/macOS). Le temps d'exécution est décompté en **minutes de calcul** (quota mensuel limité en offre Free — de l'ordre de quelques centaines de minutes à ma connaissance ; **vérifie la page de tarification**, ce quota a souvent changé).
- **Runner auto-hébergé** (gratuit, illimité, utile pour des GPU ou un réseau d'entreprise) :

```bash
# Sur ta machine/serveur, après installation de gitlab-runner :
# Settings → CI/CD → Runners → "New project runner" → récupérer le jeton (glrt-...)
gitlab-runner register \
  --url https://gitlab.com \
  --token glrt-XXXXXXXX \
  --executor docker \
  --docker-image python:3.12-slim
```

Les **tags** relient un job à un runner précis :

```yaml
entrainement-gpu:
  tags: [gpu]
  script:
    - python train.py
```

7.9 GitHub Actions vs GitLab CI/CD

	GitHub Actions	GitLab CI/CD
Fichier(s)	`.github/workflows/*.yml` (plusieurs)	`.gitlab-ci.yml` (un principal + `include`)
Ordre d'exécution	Jobs parallèles, ordre via `needs`	`stages` séquentiels, ou `needs`
Réutilisation	Actions du Marketplace (`uses:`)	`include`, `extends`, templates, CI/CD Catalog

	GitHub Actions	GitLab CI/CD
Environnement d'exécution	VM (ubuntu/windows/macos)	Conteneur Docker (`image:`) en général
Condition	`if:`	`rules:`
Expression / variable	`${{ github.ref_name }}`	`$CI_COMMIT_BRANCH`
Planification	`on: schedule` dans le YAML	Interface *Pipeline schedules*
Validation manuelle	Environments + required reviewers	`when: manual`

7.10 Déboguer un pipeline

- Logs du job : clic sur le job dans le pipeline. **Retry** pour relancer.
- **Pipeline editor → Validate** pour la syntaxe.
- Ajouter temporairement `- env | sort` ou `- ls -la` dans `script` pour inspecter l'environnement (attention : ne jamais afficher de secrets).
- Variable `CI_DEBUG_TRACE: "true"` = trace très détaillée — **peut exposer des secrets dans les logs**, à n'utiliser qu'avec prudence.
- Erreur classique : `This job is stuck because you don't have any active runners` → aucun runner disponible pour ce projet ou ces `tags`.

📝 Exercices

Exercice 7.1 — Ordre d'exécution

```yaml
stages: [build, test, deploy]

compiler:      { stage: build,  script: [echo build] }
tests-unit:    { stage: test,   script: [echo unit] }
tests-integ:   { stage: test,   script: [echo integ] }
doc:           { stage: deploy, script: [echo doc], needs: [compiler] }
publier:       { stage: deploy, script: [echo pub] }
```

a) Quels jobs tournent en parallèle ? b) Quand `doc` démarre-t-il ? Et `publier` ? c) Si `tests-integ` échoue, lesquels de `doc` et `publier` s'exécutent ?

Exercice 7.2 — Cache ou artifact ?

Classe chaque élément : a) les paquets pip téléchargés · b) le modèle `model.joblib` entraîné · c) le rapport `coverage.xml` · d) le dossier `node_modules/` · e) le site HTML à publier sur GitLab Pages.

Exercice 7.3 — Règles

Écris les `rules` d'un job `lint` qui doit tourner : - sur la **branche par défaut**, toujours ; - dans les pipelines de **merge request**, mais **seulement** si la MR modifie des fichiers `.py`, `.toml`, `.yml` / `.yaml` ou `.txt` (inutile de linter une MR qui ne touche que la documentation `.md`).

Exercice 7.4 — Job d'entraînement

Écris un job `entrainer` (stage `train`, image `python:3.12-slim`) qui : - ne tourne que dans les pipelines **planifiés** ou lancés **manuellement depuis l'interface** (bouton *Run pipeline*) ; - exécute `pip install -r requirements.txt` puis `python train.py` ; - conserve le dossier `models/` pendant 30 jours.

Exercice 7.5 — Regex de couverture

La sortie de pytest-cov se termine par `TOTAL 120 18 85%`. Quelle valeur de `coverage:` permet à GitLab d'afficher **85%** ?

Exercice 7.6 — `extends`

```yaml
.base:
  image: python:3.12-slim
  before_script:
    - pip install -r requirements.txt
  variables:
    ENV: test

tests:
  extends: .base
  before_script:
    - pip install -r requirements-dev.txt
  variables:
    DEBUG: "1"
  script:
    - python -m pytest
```

a) Quelle(s) commande(s) `before_script` exécute le job `tests` ? b) Quelles variables a-t-il ?

Exercice 7.7 — Job bloqué

Ton job affiche *This job is stuck because you don't have any active runners online or available with any of these tags assigned to them: gpu*. Explique et propose deux solutions.

🔑 Corrigés

Corrigé 7.1

a) `tests-unit` et `tests-integ` (même stage `test`). b) `doc` démarre **dès que `compiler` est terminé** grâce à `needs` (en même temps que les tests). `publier` attend la fin de **tout** le stage `test`. c) `doc` s'exécute (il ne dépend que de `compiler`) ; `publier` **ne s'exécute pas** (un job du stage précédent a échoué).

Corrigé 7.2

a) **cache** · b) **artifact** (un résultat à conserver) · c) **artifact** (rapport, `artifacts:reports`) · d) **cache** · e) **artifact** (dossier `public/` du job `pages`). Règle : *cache* = accélérer, peut disparaître ; *artifact* = résultat dont tu as besoin.

Corrigé 7.3

```yaml
lint:
  script:
    - ruff check .
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      changes:
        - "**/*.{py,toml,yml,yaml,txt}"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Les règles sont évaluées **dans l'ordre**, la première qui correspond décide. Dans une MR, `changes` compare la branche à la branche cible : le job n'est ajouté que si au moins un fichier correspondant au motif a changé. Une MR ne touchant que des `.md` ne correspond à aucune règle → job absent. Sur la branche par défaut, la 2e règle (sans `changes`) l'ajoute toujours. Les accolades `{py,toml,...}` sont un motif glob accepté par `changes`.

Corrigé 7.4

```yaml
stages: [train]

entrainer:
  stage: train
  image: python:3.12-slim
  script:
    - pip install -r requirements.txt
    - python train.py
  artifacts:
    paths:
      - models/
    expire_in: 30 days
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
    - if: $CI_PIPELINE_SOURCE == "web"
```

Corrigé 7.5

```yaml
coverage: '/TOTAL.*\s+(\d+%)$/'
```

`TOTAL` puis n'importe quoi, des espaces, et un **groupe capturé** `(\d+%)` en fin de ligne → `85%`. Les guillemets simples évitent que YAML interprète les `\`.

Corrigé 7.6

a) **Seulement** `pip install -r requirements-dev.txt` : avec `extends`, les **listes sont remplacées**, pas concaténées (le `before_script` de `.base` est écrasé). b) `ENV=test` **et** `DEBUG=1` : les **dictionnaires** (comme `variables`) sont fusionnés. (Pour combiner des listes, utiliser `!reference [.base, before_script]`.)

Corrigé 7.7

Le job a `tags: [gpu]` : seuls les runners portant le tag `gpu` peuvent le prendre, et **aucun n'est disponible** (pas enregistré, hors ligne, ou non autorisé pour ce projet). Solutions : enregistrer/redémarrer un runner avec le tag `gpu` (`gitlab-runner register ... --tag-list gpu`, ou dans l'interface du runner), ou retirer/changer le tag si le job n'a pas vraiment besoin d'un GPU.

Chapitre 8 — GitKraken : le client Git graphique avancé

8.1 C'est quoi ?

GitKraken Desktop est un **client Git graphique** (Windows, macOS **et Linux**) édité par la société GitKraken (qui fait aussi l'extension **GitLens** pour VS Code). Sa force : un **graphe de commits** très visuel et des opérations avancées (rebase, cherry-pick, résolution de conflits) en **glisser-déposer**.

	GitHub Desktop	GitKraken Desktop
Prix	Gratuit, open source	Gratuit pour les dépôts **publics** ; dépôts **privés** et fonctions avancées → offre payante (vérifie la grille tarifaire actuelle, elle change régulièrement)
Plateformes	Windows, macOS	Windows, macOS, Linux
Intégrations	GitHub	GitHub, GitLab, Bitbucket, Azure DevOps (+ Jira, etc.)
Graphe des branches	Basique	**Très riche**
Rebase interactif	Partiel (squash, réordonner)	Complet, visuel
Outil de conflit intégré	Non (passe par ton éditeur)	**Oui**
Courbe d'apprentissage	Très douce	Moyenne

> 💡 Si tu travailles avec **GitLab** (souvent en entreprise), GitKraken est un meilleur choix graphique que GitHub Desktop.

8.2 Installation et connexion

1. Télécharger sur https://www.gitkraken.com.
2. Créer un compte ou se connecter **avec GitHub / GitLab**.
3. **Preferences → Integrations** : connecter GitHub, GitLab (GitLab.com ou une instance self-managed avec son URL), Bitbucket… GitKraken peut même **générer et envoyer ta clé SSH** au service.
4. **Preferences → Profiles** : nom et email des commits (on peut avoir plusieurs profils : perso / pro).

8.3 L'interface






```
      Barre d'outils en haut : Undo · Redo · Pull · Push · Branch · Stash · Pop · Terminal
```

- Chaque **colonne colorée** est une branche ; chaque **point** un commit.
- La ligne **« // WIP »** en haut = tes modifications non commitées.
- **Palette de commandes** : `Ctrl + P` (Windows/Linux) / `Cmd + P` (macOS) → tape une action (« create branch », « push »…).
- Bouton **Terminal** : ouvre un terminal intégré dans le dépôt (avec autocomplétion Git).

8.4 Utilisation pas à pas

a) Ouvrir / cloner

File → Clone Repo (ou `Ctrl/Cmd + N`) : depuis une URL ou directement depuis GitHub/GitLab connectés. **File → Open Repo** pour un dossier local. **File → Init Repo** pour en créer un.

b) Commit

1. Clique sur la ligne **WIP**.
2. Panneau droit : **Stage File** fichier par fichier, **Stage all changes**, ou ouvre le fichier pour stager **ligne par ligne / bloc par bloc** (*Stage hunk*).
3. **Summary** + description → **Commit changes to N files**.

c) Branches

- Bouton **Branch** dans la barre d'outils → nom → Entrée (la branche est créée et *checkoutée*).
- **Double-clic** sur une branche = `git switch`.
- Clic droit sur une branche → *Rename*, *Delete*, *Set upstream*…

d) Push / Pull

- **Push** : pousse la branche courante (propose de créer la branche distante si nécessaire).
- **Pull** : le menu déroulant à côté permet de choisir *Fetch All*, *Pull (fast-forward if possible)*, *Pull (fast-forward only)*, *Pull (rebase)*.

e) Merge et rebase par glisser-déposer ⭐

Glisse l'étiquette de la branche `feature/nettoyage` **sur** l'étiquette `main` → menu : - *Merge feature/nettoyage into main* - *Rebase … onto …* - *Fast-forward …* (si possible)…

> ⚠️ Lis bien le libellé avant de cliquer : la phrase « **X onto Y** » signifie que c'est **X** qui est réécrite par-dessus Y, et « **merge X into Y** » que c'est **Y** qui reçoit le commit de merge. Le **sens** du glisser-déposer et la branche actuellement checkoutée changent les options proposées. Dans le doute, le bouton **Undo** est là.

f) Rebase interactif

Clic droit sur un commit → **Interactive Rebase** → pour chaque commit : *Pick*, *Reword* (changer le message), *Squash* (fusionner avec le précédent), *Drop* (supprimer), et glisser pour réordonner → **Start Rebase**.

g) Cherry-pick, revert, reset

Clic droit sur n'importe quel commit du graphe : - **Cherry pick commit** ; - **Revert commit** ; - **Reset to this commit** → *Soft / Mixed / Hard* (mêmes significations qu'en ligne de commande, chapitre 1.11) ; - **Create branch here**, **Create tag here**.

h) Undo / Redo 🛟

Le bouton **Undo** annule la dernière opération Git (commit, checkout, suppression de branche, reset…). Très rassurant pour apprendre. Certaines opérations comme un push ne sont pas annulables par ce bouton.

i) Résoudre un conflit

En cas de conflit, GitKraken affiche les fichiers en conflit ; clic sur un fichier → **outil de fusion intégré** à trois panneaux : version A (ta branche) / version B (l'autre) / résultat. Coche les lignes à garder (ou édite le résultat), **Save**, puis **Commit and Merge**.

j) Pull Requests / Merge Requests

Panneau gauche → **PULL REQUESTS** (`+` pour en créer une) : fonctionne avec GitHub, GitLab (MR), Bitbucket et Azure DevOps une fois l'intégration connectée. Tu peux aussi glisser une branche sur une autre et choisir *Start a pull request*.

k) Stash

Bouton **Stash** (met de côté le WIP) et **Pop** (le ré-applique). Les stashes apparaissent dans le graphe et dans le panneau gauche.

8.5 Fonctions supplémentaires (selon l'offre)

- **Workspaces** : regrouper plusieurs dépôts (ex. `api`, `pipeline-etl`, `dashboard`) et agir sur tous.
- **Launchpad** : vue centralisée des PR/issues qui t'attendent sur tous tes services.
- **GitLens** (VS Code) et **GitKraken CLI** : même éditeur, autres interfaces.

Les noms et le périmètre exact de ces fonctionnalités évoluent souvent : réfère-toi à la documentation officielle de GitKraken.

8.6 Correspondance actions ↔ commandes

GitKraken	Ligne de commande
Stage file / hunk	`git add <fichier>` / `git add -p`
Commit changes	`git commit -m "..."`
Bouton Branch	`git switch -c <branche>`
Double-clic sur une branche	`git switch <branche>`
Glisser A sur B → Merge	`git switch B && git merge A`
Glisser → Rebase	`git rebase <base>`
Interactive Rebase	`git rebase -i <commit>`

GitKraken	Ligne de commande
Cherry pick commit	`git cherry-pick <hash>`
Revert commit	`git revert <hash>`
Reset → Soft/Mixed/Hard	`git reset --soft/--mixed/--hard <hash>`
Stash / Pop	`git stash` / `git stash pop`
Fetch All	`git fetch --all`

📝 Exercices

Exercice 8.1 — Actions ↔ commandes

Donne la commande Git équivalente : a) double-clic sur la branche `dev` · b) *Stage hunk* · c) *Reset main to this commit → Mixed* · d) *Cherry pick commit* · e) *Pull (rebase)* · f) bouton *Stash*.

Exercice 8.2 — Nettoyer avant la PR

Ta branche contient 4 commits : `wip`, `wip 2`, `feat: ajoute l'export CSV`, `typo`. Tu veux **un seul** commit propre nommé `feat: ajoute l'export CSV` avant d'ouvrir ta PR. Comment faire avec GitKraken ? Quelle est la commande équivalente ?

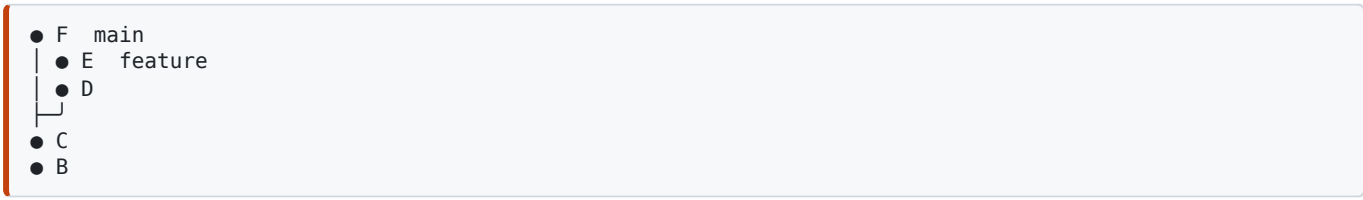
Exercice 8.3 — Soft, Mixed, Hard

Après un *Reset to this commit* sur le commit précédent, où se trouvent les modifications du commit annulé avec chaque mode ?

Exercice 8.4 — Branche supprimée par erreur

Tu as supprimé la branche locale `feature/api` (non fusionnée) dans GitKraken. Comment la récupérer, dans GitKraken puis en ligne de commande ?

Exercice 8.5 — Lire le graphe



a) Quels commits sont dans `main` mais pas dans `feature` ? Et l'inverse ? b) Quelles commandes les listent ?

Exercice 8.6 — Quel outil ?

Pour chaque besoin, propose l'outil le plus adapté (CLI, GitHub Desktop, GitKraken) : a) débutant, projets uniquement sur GitHub, sous Windows ; b) projets sur GitLab d'entreprise et sur GitHub, sous Linux, avec rebases fréquents ; c) script de CI qui doit tagger une version automatiquement.

🔑 Corrigés

Corrigé 8.1

a) `git switch dev` · b) `git add -p` (pour ce bloc) · c) `git reset --mixed <hash>` · d) `git cherry-pick <hash>` · e) `git pull --rebase` · f) `git stash`.

Corrigé 8.2

Clic droit sur le commit **parent** du premier `wip` (le dernier commit avant ta branche) → **Interactive Rebase** : - `wip` → **Pick** ; `wip 2`, `feat: ajoute l'export CSV`, `typo` → **Squash** (fusionnés dans le précédent) ; - **Reword** du commit résultant (ou édite le message proposé) → `feat: ajoute l'export CSV` ; - **Start Rebase**.

Équivalent : `git rebase -i HEAD~4` (ou `git rebase -i main`), puis `pick` / `squash` dans l'éditeur. Si la branche était déjà poussée : `git push --force-with-lease` (branche perso uniquement). Alternative sans réécriture locale : **Squash and merge** au moment du merge de la PR.

Corrigé 8.3

Mode	Modifications du commit annulé
Soft	Dans le **staging** (prêtes à être recommitées)
Mixed	Dans le **répertoire de travail**, non stagées
Hard	**Supprimées** (récupérables seulement via le reflog)

Corrigé 8.4

GitKraken : bouton **Undo** juste après la suppression. Ligne de commande : une branche non fusionnée ne se supprime qu'avec `git branch -D` (avec `-d`, Git refuse : *not fully merged*), et Git affiche alors `Deleted branch feature/api (was a1b2c3d).` Note ce hash ; sinon le reflog permet de le retrouver :

```
git reflog                          # chercher le dernier commit de feature/api
git branch feature/api a1b2c3d      # recrée la branche sur ce commit
```

Corrigé 8.5

a) Dans `main` mais pas `feature` : **F**. Dans `feature` mais pas `main` : **D et E**. (B et C sont communs.) b)

```
git log --oneline feature..main     # → F
git log --oneline main..feature     # → E, D
```

`A..B` = « les commits accessibles depuis B mais pas depuis A ».

Corrigé 8.6

a) **GitHub Desktop** (simple, intégré à GitHub, Windows). b) **GitKraken** (Linux, multi-plateformes GitHub + GitLab, rebase interactif visuel) — ou la CLI. c) **CLI** : une CI n'a pas d'interface graphique (`git tag -a v1.2.0 -m "..." && git push origin v1.2.0`).

Chapitre 9 — TP final : un projet data de A à Z

Objectif : utiliser **tous** les outils du cours sur un mini-projet d'analyse. Compte environ 2 heures.

Prérequis : Git installé, un compte GitHub **et** un compte GitLab, une clé SSH ajoutée aux deux, et les fichiers d'exemple (dossier `cours-git/exemples/` du dépôt, ou annexe / pièce jointe `exemples-cours-git.zip` de la version PDF). Dans les commandes, remplace `<chemin-vers>/cours-git/exemples` par le dossier où tu as placé ces fichiers.

Étape 1 — Git Bash / terminal : initialiser le projet

```
mkdir tp-git-data && cd tp-git-data
git init
cp -r <chemin-vers>/cours-git/exemples/projet-demo/. .
cp <chemin-vers>/cours-git/exemples/gitignore-data-science .gitignore
echo "# Projet" > README.md
git add .
git commit -m "feat: initialise le projet demo (stats + tests)"
git log --oneline
```

✔️ Vérifie : `git status` affiche « nothing to commit, working tree clean ».

Teste en local :

```
python -m venv .venv
source .venv/bin/activate          # Git Bash sous Windows : source .venv/Scripts/activate
pip install -r requirements-dev.txt
python -m pytest                    # 6 tests doivent passer
```

Étape 2 — GitHub : publier et activer la CI

```
# Crée un dépôt VIDE "tp-git-data" sur GitHub (sans README), puis :
git remote add origin git@github.com:TON_UTILISATEUR/tp-git-data.git
git push -u origin main

mkdir -p .github/workflows
cp <chemin-vers>/cours-git/exemples/github-actions/ci.yml .github/workflows/ci.yml
git add .github/workflows/ci.yml
git commit -m "ci: ajoute GitHub Actions (lint + tests)"
git push
```

✔️ Vérifie : onglet **Actions** → le workflow « CI Python » est vert (1 job lint + 3 jobs test).

Étape 3 — GitHub Desktop : une fonctionnalité via Pull Request

1. **File → Add local repository** → `tp-git-data`.
2. **New branch** : `feature/ecart-type`.
3. Dans `src/demo/stats.py`, ajoute :

`python def ecart_type(valeurs: Iterable[float | None]) -> float: """Écart-type de population en ignorant les valeurs manquantes.""" propres = nettoyer(valeurs) if not propres: raise ValueError("Impossible de calculer l'écart-type d'une liste vide") m = sum(propres) / len(propres) return math.sqrt(sum((x - m) ** 2 for x in propres) / len(propres))`

et dans `tests/test_stats.py` (pense à ajouter `ecart_type` à l'import) :

`python def test_ecart_type(): assert ecart_type([2, 4, 4, 4, 5, 5, 7, 9]) == 2.0`

> ✔️ Contrôle du calcul : moyenne = 40/8 = 5 ; somme des carrés des écarts = 9+1+1+1+0+0+4+16 = 32 ; 32/8 = 4 ; √4 = **2.0**.

4. Commit `feat: ajoute l'écart-type` → **Publish branch** → **Create Pull Request**.
5. Sur GitHub : attends que la CI soit verte → **Squash and merge** → *Delete branch*.
6. Dans Desktop : bascule sur `main` → **Pull origin**.

Étape 4 — Provoquer et résoudre un conflit (GitKraken)

1. Ouvre le dépôt dans **GitKraken**.
2. Crée la branche `feature/readme-a`, modifie la 1re ligne du `README.md` en `# Projet A`, commit.
3. Reviens sur `main`, crée `feature/readme-b`, modifie la même ligne en `# Projet B`, commit.
4. Reviens sur `main`, merge `feature/readme-a` (glisser-déposer) → OK.
5. Merge `feature/readme-b` → **conflit** → utilise l'outil de fusion intégré, garde `# Projet A et B`, *Save* → *Commit and Merge*.
6. **Push**.

✔️ Vérifie dans le graphe : `feature/readme-a` a été intégrée en **fast-forward** (`main` n'avait pas bougé, aucun commit de merge), alors que `feature/readme-b` rejoint `main` par un **commit de merge** à deux parents. C'est exactement la différence expliquée au chapitre 1.8.

Équivalent en ligne de commande, pour comprendre ce que GitKraken a fait :

```
git switch main && git merge feature/readme-a
git merge feature/readme-b      # CONFLICT
# éditer README.md, retirer les marqueurs <<<<<<< ======= >>>>>>>
git add README.md && git commit
```

Étape 5 — GitLab : même projet, autre plateforme + GitLab CI/CD

```
# Crée un projet VIDE "tp-git-data" sur GitLab, puis :
git remote add gitlab git@gitlab.com:TON_UTILISATEUR/tp-git-data.git
cp <chemin-vers>/cours-git/exemples/gitlab-ci/.gitlab-ci.yml .gitlab-ci.yml
git add .gitlab-ci.yml
git commit -m "ci: ajoute le pipeline GitLab"
git push gitlab main
git push origin main            # garder GitHub synchronisé
```

✔️ Vérifie : **Build → Pipelines** → stages *lint → test (×3) → deploy_staging*.

Puis, crée une Merge Request en une commande :

```
git switch -c feature/doc
echo "Voir le cours Git pour les détails." >> README.md
git commit -am "docs: complète le README"
git push gitlab feature/doc -o merge_request.create -o merge_request.target=main
```

✔️ Vérifie : la MR affiche le pipeline et le **pourcentage de couverture**.

Étape 6 — Release (tag) et déploiement manuel

```
git switch main && git pull gitlab main
git tag -a v1.0.0 -m "Première version"
git push gitlab v1.0.0
git push origin v1.0.0
```

✔️ Vérifie sur GitLab : un pipeline de tag apparaît avec le job **deploy_production** en attente (▶). Clique pour le lancer, puis regarde **Operate → Environments**.

Étape 7 — Annulations (filet de sécurité)

```
echo "erreur" >> src/demo/stats.py
git restore src/demo/stats.py            # annule la modif

git commit --allow-empty -m "commit raté"
git reset --soft HEAD~1                  # défait le commit (local, non poussé)

echo "ligne à annuler" >> README.md
git commit -am "docs: ligne à annuler"
git revert --no-edit HEAD                # crée un commit inverse : méthode sûre même si c'est déjà
poussé
git log --oneline -3                     # tu vois le commit ET son "Revert ..."
git reflog | head -5                     # l'historique de tous tes déplacements de HEAD
```

⚠️ `git revert` sur un **commit de merge** exige de préciser le parent à garder : `git revert -m 1 <hash>` (1 = la branche sur laquelle on a mergé, en général `main`).

🎓 Grille d'auto-évaluation

Je sais…	✅
expliquer working directory / staging / dépôt local / distant	☐
créer une branche, commiter, pousser, ouvrir une PR/MR	☐
résoudre un conflit à la main et avec un outil graphique	☐
choisir entre `merge`, `rebase`, `revert`, `reset`	☐
écrire un workflow GitHub Actions avec matrice et secrets	☐
écrire un `.gitlab-ci.yml` avec stages, rules, artifacts, job manuel	☐
utiliser GitHub Desktop et GitKraken et dire quelle commande ils exécutent	☐

📝 Exercices bonus

À faire sur le dépôt du TP.

Exercice 9.1 — Seuil de couverture

Fais **échouer la CI** (GitHub et GitLab) si la couverture des tests passe sous 90 %. Comment le vérifier en local ?

Exercice 9.2 — Nouvelle fonction en « TDD »

Sur une branche `feature/etendue`, ajoute une fonction `etendue(valeurs)` (max − min, valeurs manquantes ignorées, `ValueError` si la liste est vide). **Écris d'abord le test**, constate qu'il échoue, puis implémente. Ouvre une PR (GitHub) **et** une MR (GitLab).

Exercice 9.3 — Trouver le commit fautif avec `git bisect`

Un collègue a cassé `moyenne` quelque part dans les 20 derniers commits, et les tests échouent. Utilise `git bisect` pour trouver **automatiquement** le commit coupable.

🔑 Corrigés

Corrigé 9.1

Ajouter l'option `--cov-fail-under=90` à pytest, dans `.github/workflows/ci.yml` **et** `.gitlab-ci.yml` :

```
python -m pytest --cov=demo --cov-report=term --cov-report=xml --cov-fail-under=90
```

pytest renvoie alors un code de sortie non nul si la couverture totale est < 90 %, ce qui fait échouer le job. Vérification locale : commenter temporairement un test, relancer la commande → message `FAIL Required test coverage of 90% not reached` et code de sortie ≠ 0 (`echo $?`).

Corrigé 9.2

```
git switch main && git pull
git switch -c feature/etendue
```

Test d'abord (`tests/test_stats.py` , ajouter `etendue` à l'import) :

```
def test_etendue():
    assert etendue([3, None, 10, 1]) == 9


def test_etendue_vide():
    with pytest.raises(ValueError):
        etendue([None])
```

`python -m pytest` → **échec** (`ImportError` : la fonction n'existe pas encore). Puis l'implémentation dans `src/demo/stats.py` :

```
def etendue(valeurs: Iterable[float | None]) -> float:
    """Écart entre la valeur maximale et minimale, valeurs manquantes ignorées."""
    propres = nettoyer(valeurs)
    if not propres:
        raise ValueError("Impossible de calculer l'étendue d'une liste vide")
    return max(propres) - min(propres)
```

```
python -m pytest                      # tout passe
git add -A && git commit -m "feat: ajoute l'étendue"
git push -u origin feature/etendue    # puis "Compare & pull request" sur GitHub
git push -u gitlab feature/etendue -o merge_request.create -o merge_request.target=main
```

Corrigé 9.3

```
git bisect start
git bisect bad                # le commit actuel est cassé
git bisect good HEAD~20       # celui-ci fonctionnait
git bisect run python -m pytest -q   # Git teste automatiquement : code 0 = bon, sinon = mauvais
# → "<hash> is the first bad commit"
git bisect reset              # revenir là où tu étais
```

Git procède par **dichotomie** : pour 20 commits, environ 5 tests suffisent ($\log_2 20 \approx 4{,}3$).

Annexe — Code source complet des exemples

Ces fichiers sont ceux utilisés dans les chapitres 5, 7 et 9. Ils sont aussi **joints au PDF** (pièce jointe `exemples-cours-git.zip`, visible dans le panneau « Pièces jointes » d'Adobe Reader, Firefox, Okular…) : préfère cette version, car un copier-coller depuis un PDF peut abîmer l'indentation, fatale en Python et en YAML.

Arborescence à recréer :

```
exemples/
├── projet-demo/
│   ├── pyproject.toml
│   ├── requirements-dev.txt
│   ├── src/demo/__init__.py
│   ├── src/demo/stats.py
│   └── tests/test_stats.py
├── github-actions/ci.yml            → à copier dans .github/workflows/
├── github-actions/rapport-hebdo.yml → à copier dans .github/workflows/
├── gitlab-ci/.gitlab-ci.yml         → à copier à la racine du dépôt
└── gitignore-data-science           → à renommer en .gitignore
```

Vérifier en local :

```
cd exemples/projet-demo
python -m venv .venv
source .venv/bin/activate      # Git Bash (Windows) : source .venv/Scripts/activate
pip install -r requirements-dev.txt
ruff check .                   # attendu : All checks passed!
python -m pytest --cov=demo    # attendu : 6 passed, couverture 100 %
```

`exemples/projet-demo/pyproject.toml`

```
[project]
name = "demo"
version = "0.1.0"
requires-python = ">=3.10"

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]

[tool.ruff]
line-length = 100
```

`exemples/projet-demo/requirements-dev.txt`

```
pytest>=8
pytest-cov>=5
ruff>=0.5
```

`exemples/projet-demo/src/demo/__init__.py`

```
"""Petit paquet de démonstration utilisé par les exemples CI/CD du cours."""
```

exemples/projet-demo/src/demo/stats.py

```python
"""Fonctions statistiques simples, volontairement sans dépendance externe."""

from __future__ import annotations

import math
from collections.abc import Iterable


def nettoyer(valeurs: Iterable[float | None]) -> list[float]:
    """Retire les valeurs manquantes (None) et les NaN."""
    resultat = []
    for v in valeurs:
        if v is None:
            continue
        if isinstance(v, float) and math.isnan(v):
            continue
        resultat.append(float(v))
    return resultat


def moyenne(valeurs: Iterable[float | None]) -> float:
    """Moyenne arithmétique en ignorant les valeurs manquantes."""
    propres = nettoyer(valeurs)
    if not propres:
        raise ValueError("Impossible de calculer la moyenne d'une liste vide")
    return sum(propres) / len(propres)


def mediane(valeurs: Iterable[float | None]) -> float:
    """Médiane en ignorant les valeurs manquantes."""
    propres = sorted(nettoyer(valeurs))
    n = len(propres)
    if n == 0:
        raise ValueError("Impossible de calculer la médiane d'une liste vide")
    milieu = n // 2
    if n % 2 == 1:
        return propres[milieu]
    return (propres[milieu - 1] + propres[milieu]) / 2
```

exemples/projet-demo/tests/test_stats.py

```python
import math

import pytest

from demo.stats import mediane, moyenne, nettoyer


def test_nettoyer_retire_none_et_nan():
    assert nettoyer([1, None, 2.0, math.nan, 3]) == [1.0, 2.0, 3.0]


def test_moyenne():
    assert moyenne([1, 2, 3, None]) == 2.0


def test_mediane_impaire():
    assert mediane([3, 1, 2]) == 2


def test_mediane_paire():
    assert mediane([4, 1, 3, 2]) == 2.5


@pytest.mark.parametrize("fonction", [moyenne, mediane])
def test_liste_vide_leve_une_erreur(fonction):
    with pytest.raises(ValueError):
        fonction([None, math.nan])
```

exemples/github-actions/ci.yml

```yaml
# Emplacement réel : .github/workflows/ci.yml (à la racine du dépôt GitHub)
# Hypothèse : le contenu de projet-demo/ est à la racine du dépôt.
name: CI Python

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:          # bouton "Run workflow" dans l'onglet Actions

permissions:
  contents: read              # principe du moindre privilège

concurrency:                  # annule un run en cours si un nouveau push arrive sur la même branche
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install ruff
      - run: ruff check .

  test:
    needs: lint               # ne démarre que si "lint" a réussi
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12", "3.13"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip
          cache-dependency-path: requirements-dev.txt
      - name: Installer les dépendances
        run: pip install -r requirements-dev.txt
      - name: Lancer les tests
        run: python -m pytest --cov=demo --cov-report=term --cov-report=xml
      - name: Publier le rapport de couverture
        if: matrix.python-version == '3.12'
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage.xml
```

exemples/github-actions/rapport-hebdo.yml

```yaml
# Emplacement réel : .github/workflows/rapport-hebdo.yml
# Exemple data : exécuter un script chaque lundi à 6h UTC avec un secret.
name: Rapport hebdomadaire

on:
  schedule:
    - cron: "0 6 * * 1"       # minute heure jour-du-mois mois jour-de-semaine (UTC)
  workflow_dispatch:

jobs:
  rapport:
    runs-on: ubuntu-latest
    steps:
```

```yaml
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install pandas
      - name: Générer le rapport
        env:
          DB_URL: ${{ secrets.DB_URL }}   # défini dans Settings > Secrets and variables > Actions
        run: python scripts/rapport.py --sortie rapport.csv
      - uses: actions/upload-artifact@v4
        with:
          name: rapport-${{ github.run_number }}
          path: rapport.csv
          retention-days: 30
```

exemples/gitlab-ci/.gitlab-ci.yml

```yaml
# Emplacement réel : .gitlab-ci.yml (à la racine du dépôt GitLab)
# Hypothèse : le contenu de projet-demo/ est à la racine du dépôt.

stages:
  - lint
  - test
  - deploy

default:
  image: python:3.12-slim

variables:
  PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"

# Le pipeline tourne pour les merge requests et pour la branche par défaut,
# sans doublon quand une MR est ouverte.
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_TAG

cache:
  key:
    files:
      - requirements-dev.txt
  paths:
    - .cache/pip

lint:
  stage: lint
  script:
    - pip install ruff
    - ruff check .

test:
  stage: test
  parallel:
    matrix:
      - PYTHON_VERSION: ["3.11", "3.12", "3.13"]
  image: python:${PYTHON_VERSION}-slim
  script:
    - pip install -r requirements-dev.txt
    - python -m pytest --cov=demo --cov-report=term --cov-report=xml
  coverage: '/TOTAL.*\s+(\d+%)$/'
  artifacts:
    when: always
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml
    expire_in: 1 week
```

```yaml
deploy_staging:
  stage: deploy
  script:
    - echo "Déploiement sur l'environnement de staging…"
  environment:
    name: staging
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

deploy_production:
  stage: deploy
  script:
    - echo "Déploiement en production de la version $CI_COMMIT_TAG"
  environment:
    name: production
  rules:
    - if: $CI_COMMIT_TAG
      when: manual            # bouton ▶ à cliquer dans l'interface
```

exemples/gitignore-data-science

```
# À renommer en .gitignore à la racine du projet

# Python
__pycache__/
*.py[cod]
.venv/
venv/
*.egg-info/
.pytest_cache/
.coverage
coverage.xml
.ruff_cache/

# Jupyter
.ipynb_checkpoints/

# Données et modèles (trop lourds / confidentiels -> DVC, Git LFS ou stockage cloud)
data/raw/
data/processed/
*.parquet
*.csv
!data/sample/*.csv
models/*.pkl
models/*.joblib
mlruns/

# Secrets
.env
*.pem
credentials*.json

# Système / IDE
.DS_Store
Thumbs.db
.idea/
.vscode/
```